

Faculdade de Tecnologia de Rio Preto

Trabalho #3: ULA 4 Bits & 4 Operações

Unidade Lógica Aritmética de 4 bits e 4 operações (2 Aritméticas e 2 Lógicas)

Aluno: Luiz Francisco Neves Mendes – **RA:** 1210482522040

Turma: Tarde

Disciplina: Arquitetura e Organização de Computador

Curso: Tecnologia em Análise e Desenvolvimento de Sistemas

Professor: Me José A. A. Viana

São José do Rio Preto, 24 de Setembro de 2025.

Sumário

INTRODUÇÃO.....	5
OBJETIVO.....	6
GLOSSÁRIO.....	7
Valores Digitais.....	7
Representação Digital.....	7
Junção dos Bits.....	7
Agrupamento de Bits.....	7
Sistema binário.....	7
Pesos em Bits.....	8
Sistema hexadecimal.....	8
Termos Estruturais / Conceituais.....	9
Bit (Binary Digit).....	9
Nibble (4 bits).....	9
Byte (8 bits).....	9
MSB (Most Significant Bit).....	9
LSB (Least Significant Bit).....	9
Input (Entrada).....	9
Output (Saída).....	10
Barramento.....	10
Sinal lógico / Nível lógico (0 e 1).....	10
Propagação de sinal.....	10
Delay.....	10
Termos Lógicos / Matemáticos Gerais.....	11
Expressão Booleana / Expressão lógica.....	11
Função lógica.....	11
Operação lógica.....	11
Tabela Verdade.....	11
LSB (Least Significant Bit).....	11
MSB (Most Significant Bit).....	12
MinTermos (MinTerm).....	12
MaxTermos (MaxTerm).....	12
Simplificação lógica.....	12
Equivalência lógica.....	12
1 — FUNDAMENTOS.....	13
1.1 — Glossário dos Fundamentos.....	13
1.1.1 — Termos de Circuitos.....	13
1.1.1.1 — Circuito combinacional.....	13
1.1.1.2 — Circuito sequencial.....	13
1.1.1.3 — Hierarquia de circuitos.....	13
1.1.1.4 — Bloco macro (Macro-Block ou apenas Macro).....	13
1.1.1.5 — Modularidade de circuitos.....	13
1.1.1.6 — Componente / Subcircuito.....	14

1.1.1.7 — Componentes, Módulos e Macros.....	14
1.1.2 — Termos sobre as Operações.....	15
1.1.2.1 — Carry-in (Cin).....	15
1.1.2.2 — Carry-out (Cout).....	15
1.1.2.3 — Overflow.....	15
1.1.2.4 — Borrow.....	15
1.1.2.5 — Ripple Carry.....	15
1.1.2.6 — Operação aritmética e lógica.....	15
1.1.2.7 — Modo de Operação.....	16
1.1.2.8 — Circuitos “Full” e “Half”.....	16
1.1.2.9 — Flags.....	16
1.1.3 — Termos relacionados à Simulação.....	17
1.1.3.1 — Simulação digital.....	17
1.1.3.2 — Wire.....	17
1.1.3.3 — Pin.....	17
1.1.3.4 — Splitter.....	17
1.1.3.5 — Display Hex.....	18
1.1.3.6 — Clock.....	18
1.1.3.7 — Label.....	19
1.1.3.8 — Tunnel.....	19
1.2 — Cálculos e Conversões.....	20
1.2.1 — Dígitos binários.....	20
1.2.2 — Agrupamento em Nibbles.....	20
1.2.3 — Por que multiplicar por 16.....	20
1.2.4 — Conversões no Sistema Hexadecimal.....	21
1.3 — Unidade Lógica Aritmética (ULA).....	22
1.4 — Fundamentos dos Circuitos Lógicos Combinacionais Digital.....	23
1.4.1 — Portas lógicas.....	23
1.4.1.1 — Vida real (CIs).....	23
1.4.1.2 — Buffer.....	24
1.4.1.3 — Porta NOT.....	25
1.4.1.4 — Porta AND.....	26
1.4.1.5 — Porta OR.....	27
1.4.1.6 — Porta XOR.....	28
1.4.1.7 — Porta NAND.....	29
1.4.1.8 — Porta NOR.....	30
1.4.1.9 — Porta XNOR.....	31
1.4.1.10 — Resumo dos símbolos em expressões.....	32
1.5 — Ferramenta de Simulação: Logisim.....	33
1.5.1 — Uso do Logisim.....	35
2 — MÉTODO DE DESENVOLVIMENTO.....	36
2.1 — Processo de Construção dos Circuitos.....	36
2.2 — Testes e Validações do Projeto.....	37
3 — DESENVOLVIMENTO DOS CIRCUITOS DA ULA.....	38

3.1 — Circuito: Buffer de 4 bits.....	38
3.1.1 — Expressão Matemática.....	38
3.1.2 — Tabela Verdade.....	38
3.1.3 — Circuito Lógico Padrão.....	38
3.1.4 — Bloco Macro Padrão.....	39
3.1.5 — Circuito Lógico Customizado.....	39
3.1.6 — Bloco Macro Customizado.....	39
3.2 — Circuito: Nand 4 bits (NE).....	40
3.2.1 — Expressão Matemática.....	40
3.2.2 — Tabela Verdade/Funcional.....	40
3.2.3 — Circuito Lógico Padrão.....	40
3.2.4 — Bloco Macro Padrão.....	41
3.2.5 — Circuito Lógico Customizado.....	41
3.2.6 — Bloco Macro Customizado.....	41
3.3 — Circuito: Adder 4 bits (Somador).....	42
3.3.1 — Adder Half (AdderH).....	43
3.3.1.1 — Expressão Matemática.....	43
3.3.1.2 — Tabela Verdade.....	43
3.3.1.3 — Circuito Lógico Padrão.....	43
3.3.1.4 — Bloco Macro Padrão.....	43
3.3.1.5 — Circuito Lógico Customizado.....	44
3.3.1.6 — Bloco Macro Customizado.....	44
3.3.2 — Adder 1 bit (Adder1b).....	45
3.3.2.1 — Expressão Matemática.....	45
3.3.2.2 — Tabela Verdade.....	45
3.3.2.3 — Circuito Lógico Padrão.....	45
3.3.2.4 — Bloco Macro Padrão.....	46
3.3.2.3 — Circuito Lógico Customizado.....	46
3.3.2.4 — Bloco Macro Customizado.....	46
3.3.3 — Aplicando no Adder 4 bits (Ripple-Carry).....	47
3.3.3.1 — Expressão Matemática.....	47
3.3.3.2 — Tabela Verdade.....	47
3.3.3.3 — Circuito Lógico Padrão.....	47
3.3.3.4 — Bloco Macro Padrão.....	48
3.3.3.5 — Circuito Lógico Customizado.....	48
3.3.3.6 — Bloco Macro Customizado.....	48
3.4 — Circuito: Subtractor 4 bits (Subtração).....	49
3.4.1 — Expressão Matemática.....	49
3.4.2 — Tabela Verdade.....	49
3.4.3 — Circuito Lógico Padrão.....	50
3.4.4 — Bloco Macro Padrão.....	50
3.4.5 — Circuito Lógico Customizado.....	50
3.4.6 — Bloco Macro Customizado.....	51
3.5 — Circuito: Decoder (Decodificador).....	52

3.5.1 — Expressão Matemática.....	52
3.5.2 — Tabela Verdade.....	52
3.5.3 — Circuito Lógico Padrão.....	53
3.5.4 — Bloco Macro Padrão.....	53
3.5.5 — Circuito Lógico Customizado.....	53
3.5.6 — Bloco Macro Customizado.....	54
3.6 — Circuito: Mux 4 bits (Multiplexer) (Multiplexador).....	55
3.6.1 — Mux 1 bit (Mux1b).....	55
3.6.1.1 — Expressão Matemática.....	55
3.6.1.2 — Tabela Verdade.....	55
3.6.1.3 — Circuito Lógico Padrão.....	56
3.6.1.4 — Bloco Macro Padrão.....	56
3.6.1.5 — Circuito Lógico Customizado.....	56
3.6.1.6 — Bloco Macro Customizado.....	57
3.6.2 — Aplicando no MUX 4 bits (Encadeamento).....	58
3.6.2.1 — Expressão Matemática.....	58
3.6.2.2 — Tabela Verdade.....	58
3.6.2.3 — Circuito Lógico Padrão.....	58
3.6.2.4 — Bloco Macro Padrão.....	59
3.6.2.5 — Circuito Lógico Customizado.....	59
3.6.2.6 — Bloco Macro Customizado.....	60
4 — MONTAGEM DA ULA.....	61
4.1 — Resultado final dos Circuitos e Macros.....	61
4.1.1 — Circuito Lógico Padrão.....	61
4.1.2 — Bloco Macro Padrão.....	62
4.1.3 — Circuito Lógico Customizado.....	62
4.1.4 — Bloco Macro Customizado.....	63
4.2 — Demonstração da ULA.....	64
CONCLUSÃO.....	65
Referências Bibliográficas.....	66

INTRODUÇÃO

A evolução da computação digital depende diretamente do desenvolvimento de circuitos lógicos cada vez mais eficientes e estruturados, como as Unidades Lógicas e Aritméticas (ULAs), fundamentais para o funcionamento de qualquer processador. É nesses circuitos que ocorrem os cálculos e as operações lógicas essenciais, realizados de forma direta e previsível, constituindo o núcleo operacional de uma Unidade Central de Processamento (CPU).

Propõe-se o desenvolvimento de uma ULA de 4 bits utilizando a ferramenta Logisim, contendo operações aritméticas e lógicas. Será projetado e construído desde dos operadores necessários, a organização dos componentes até a integração de um componente capaz de simular o funcionamento de ULA.

OBJETIVO

O objetivo deste trabalho é desenvolver e simular uma Unidade Lógica e Aritmética (ULA) de 4 bits, em um software capaz de simular o funcionamento lógico. Esta ULA irá conter 4 operações, sendo duas aritméticas e 2 duas lógicas, estas sendo: Soma e Subtração, e Nand e Buffer. Além de outros componentes essenciais para o funcionamento, como decodificador e Multiplexador. Todas esses componentes e operações, serão construídas de forma modular e individualmente, e em seguida, integradas seguindo o modelo de organização de processadores.

O projeto, também busca a compreensão do funcionamento lógico por trás dos diversos circuitos, desde de técnicas para melhorar o entendimento, a eficiência até permitir uma melhor estética para os componentes.

Tudo isso, está sendo planejado para analisar pontos críticos de um processador, como técnicas de overflow, carries e sinalizações — conhecidas como flags — para permitir uma futura expansão do projeto, para a construção de uma CPU simples, porém, por conta do planejado, espera-se que a CPU apresente uma ULA com mais operações.

GLOSSÁRIO

Este trabalho possui um glossário dos termos que serão citados durante o desenvolvimento. Esses serão citados durante o desenvolvimento, então se espera o conhecimento prévio desses.

Dito isso, muitos significados de uma vez, podem ser excessivos, por isso, em caso do pouco conhecimento sobre circuitos digitais, recomenda-se entender os tópicos: [Valores Digitais](#), [Termos Estruturais / Conceituais](#) e [Termos Lógicos / Matemáticos Gerais](#).

Em seguida para: [Glossário dos Fundamentos](#), em [FUNDAMENTOS](#).

Para a tratar com valores: [Cálculos e Conversões](#), e continuar a leitura a partir do tópico: [Unidade Lógica Aritmética \(ULA\)](#).

Valores Digitais

Representação Digital

Em circuitos digitais, as informações são representadas por dois estados possíveis, denominados níveis lógicos: 0 — nível lógico baixo e 1 — nível lógico alto. Estes estados correspondem a níveis de tensão, definidas pela tecnologia do circuito. Em suma, 1 representa a passagem de energia, enquanto 0, não.

Junção dos Bits

Se refere à junção de bits, combinando seus valores para permitir valores maiores, onde, os mais à direita têm pesos menores do que os da esquerda, peso é a quantidade de valor, que aquele bit vale, de acordo com seu deslocamento na linha. Por exemplo, sempre da **direita** para a **esquerda**, o primeiro tem o peso mais leve, enquanto, mais distante do primeiro, maior seu peso. Essa regra é a mais usada, e permite diferentes interpretações dos diferentes sistemas para leitura e interpretação de bits.

Agrupamento de Bits

Agrupamento de Bits, é o ato de agrupar os bits, eles também aumentam os valores dos bits de acordo com o deslocamento, porém, permitem uma melhor leitura dos dados, ao menos para o humano. Muitos desses, recebem “apelidos”, 1 bit e 2 bits, não têm nomes próprios, mas 4 bits formam um Nibble, e 8 bits um Byte.

Sistema binário

Esse é o sistema mais conhecido para a representação de números em ambientes digitais. Ele utiliza apenas os dígitos 0 e 1, que representam valores lógicos distintos. A interpretação de um número binário depende da quantidade de bits no agrupamento, pois cada posição possui um **peso**, determinado pela potência de 2 correspondente à sua posição.

Os pesos são atribuídos da direita para a esquerda, iniciando em 2^0 . Assim, a primeira casa vale 1, a segunda vale 2, a terceira 4, a quarta 8, e assim por diante.

É importante destacar que, ao apresentar valores numéricos em qualquer base — seja em textos, diagramas, imagens ou circuitos — deve-se indicar o sufixo da base, principalmente quando pode haver ambiguidades. Exemplos:

- 1011_{10} (= mil e onze em decimal);
- 1011_2 (= 11 em binário);
- 13_8 (= 11 em octal);
- B_{16} (= 11 em hexadecimal).

Pesos em Bits

O peso é o valor atribuído a cada bit de acordo com sua posição dentro de um número. Em sistemas de numeração posicional, cada casa possui um valor proporcional à sua base elevada ao índice da posição.

No sistema binário, os pesos crescem de acordo com as potências de 2, sempre da direita para a esquerda. Em agrupamentos de quatro bits, por exemplo, os pesos são **8, 4, 2 e 1**. Assim, cada bit contribui de forma diferente para o valor total do número, dependendo de sua posição.

Esse conceito será melhor explicado na seção “**1.1 — Cálculos e Conversões com dígitos binários**”, onde os pesos serão aplicados em operações práticas.

Sistema hexadecimal

O sistema hexadecimal utiliza 16 símbolos para representar valores numéricos:

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E e F.

Os símbolos de A a F correspondem aos valores decimais de 10 a 15.

Este sistema tem uma correspondência direta com o sistema binário:

- 4 bits formam um nibble,
- e cada nibble equivale exatamente a um dígito hexadecimal.

Isso torna o hexadecimal extremamente útil para leitura, depuração e documentação de circuitos digitais, reduz longas sequências binárias a representações mais compactas e legíveis. Inclusive, é uma famosa forma de representar cores em computadores, como “FF0000”, que, em RGB: 255, 0, 0 — equivalente ao vermelho.

Exemplos de uso:

- $1011_2 = B_{16} = 11_{10}$
- $1111_2 = F_{16} = 15_{10}$
- $1100\ 1010\ 1111\ 1110_2 = \text{CAFE}_{16} = 51966_{10}$

Assim como no binário, recomenda-se sempre indicar explicitamente a base — usando subscrito ou sufixo — para evitar interpretações ambíguas em diagramas e textos técnicos.

Termos Estruturais / Conceituais

Bit (Binary Digit)

Refere-se à menor unidade de informação em sistemas digitais. Capaz de assumir dois valores: 0 ou 1, que representam estados lógicos de um circuito digital.

Importante citar que — bits são diferentes de valores — quando falamos na fórmula binária de ver os bits, eles sempre estarão representados por 0 e 1, porém, o valor que eles representam, variam de acordo com suas combinações, um único bit, pode assumir os valores de 0 e 1.

Nibble (4 bits)

Conjunto formado por quatro bits agrupados, que também equivale a um dígito em hexadecimal. É utilizado para representar valores, que podem variar entre 0 à 15. Os circuitos desenvolvidos, serão para montar uma ULA de 4 bits, e em diversos momentos será possível visualizar sequências de quatros 0 ou 1.

Byte (8 bits)

Conjunto formado por oito bits agrupados, equivalente à 2 nibbles ou 2 dígitos hexadecimais. É utilizado para representar valores, que podem variar entre 0 e 255, dependendo das combinações realizadas

MSB (Most Significant Bit)

Refere-se ao bit mais significativo dentro de uma sequência binária. É o bit de maior peso, normalmente localizado à esquerda do agrupamento. Em representações binárias, o MSB é o responsável por indicar a maior parcela do valor numérico daquele conjunto de bits.

LSB (Least Significant Bit)

É o bit menos significativo da sequência binária, localizado na posição mais à direita do agrupamento. Possui o menor peso dentro da representação numérica. O LSB é frequentemente utilizado em operações como incremento, verificação de paridade e manipulações de menor granularidade em circuitos digitais.

Input (Entrada)

Corresponde a qualquer valor lógico, dado ou sinal fornecido a um circuito digital. As entradas definem as condições iniciais que permitirão ao circuito produzir um resultado. Em diagramas digitais, são representadas como pinos ou portas que recebem sinais vindos de outros componentes.

Output (Saída)

É o resultado gerado por um circuito digital após o processamento de suas entradas. Representa o valor lógico final produzido pela operação realizada. As saídas podem ser visualizadas como pinos de saída, LEDs, displays ou ligações com outros módulos.

Barramento

É um conjunto organizado de múltiplas linhas de sinal que transportam vários bits simultaneamente. Um barramento permite o tráfego paralelo de dados entre diferentes partes do circuito, facilitando a conexão de somadores, multiplexadores, entre outros dispositivos. Em diagramas, é representado como uma linha mais espessa indicando múltiplos bits agrupados.

Em muitos dos circuitos desenvolvidos os barramentos serão vistos como um único pino de input, os quais, podem transportar diferentes tamanhos de bits, desde apenas um único bit, até um nibble. Caso necessário, os bits desses nibbles, serão divididos, usando dispositivos oferecidos pelas ferramentas, os quais permitem dividir um input de 4 bits, em 4 fios de 1 bit cada, mantendo seus valores sincronizados com a entrada. Essa escolha, é puramente estética e, é uma forma de mostrar o que a ferramenta utilizada pode fazer, mas ainda sim, serão mantidos os modelos recomendados, para melhor didática e entendimento.

Sinal lógico / Nível lógico (0 e 1)

É a representação elétrica do estado de um circuito digital. O nível lógico 0 (ou baixo) e o nível lógico 1 (ou alto) correspondem a faixas específicas de tensão dependendo da tecnologia do circuito. Em ferramentas, esses valores são tratados apenas como 0 e 1, determinando o comportamento das portas lógicas e dos componentes digitais.

Propagação de sinal

Refere-se ao percurso que um valor lógico realiza ao atravessar um circuito, passando por fios, portas lógicas e componentes até atingir a saída. A propagação está relacionada ao tempo que o circuito leva para transferir e processar um sinal, influenciando o comportamento temporal e a estabilidade das operações digitais.

Delay

É o intervalo de tempo necessário para que uma alteração em uma entrada produza o efeito correspondente nas saídas do circuito. O delay ocorre devido às características físicas das portas lógicas, fios e elementos internos. Em simulações digitais simplificadas, pode ser visualizado quando uma mudança de valor não é refletida instantaneamente no resultado, que podem ou não serem percebidas, dependendo de diversos fatores de desempenho.

Termos Lógicos / Matemáticos Gerais

Expressão Booleana / Expressão lógica

É uma forma simbólica de representar o comportamento de um circuito digital usando variáveis (A, B, X...), operadores lógicos (AND, OR, NOT...) e constantes (0 e 1).

Uma expressão booleana descreve exatamente como uma saída depende das entradas. Por exemplo, a expressão: $X = A \text{ AND } B$, representa um circuito onde a saída **X** só é 1 quando **A** e **B** são 1. Expressões booleanas servem como “linguagem matemática” para modelar circuitos lógicos, por isso, podem ter diferentes notações em suas representações.

Função lógica

É a relação completa entre entradas e saídas de um circuito digital, podendo ser descrita por uma expressão booleana, tabela verdade ou circuitos esquemáticos. Uma função lógica determina qual saída é produzida para cada combinação possível de entradas. Exemplo são a função AND, a função OR, função XOR, etc. A “função” é o comportamento, não o circuito em si.

Operação lógica

É a ação individual realizada por portas lógicas básicas, como AND, OR, NOT, NAND, NOR, XOR, XNOR. Cada operação define regras específicas de como combinar bits de entrada para produzir um bit de saída. Operações lógicas são os “blocos fundamentais” usados para construir expressões booleanas e funções lógicas.

Tabela Verdade

É uma tabela que enumera todas as combinações possíveis das entradas e mostra o valor correspondente da saída. Muitas das vezes, as combinações podem gerar tabelas gigantes — como multiplexadores —, então é possível apresentar as tabelas em um resumo funcional de suas funcionalidades, essas geralmente têm “i” que indica para cada bit, além de outros indicadores que serão apresentados como guia.

É essencial para analisar ou validar o comportamento de uma função lógica. Por exemplo, para duas entradas (A e B), a tabela verdade terá 4 linhas (00, 01, 10, 11).

LSB (Least Significant Bit)

É o **bit menos significativo** de uma sequência binária, é o bit mais à direita. Ele representa o **menor peso** na representação do número ($2^0 = 1$). Ele é importante para: operações de soma, manipulação de deslocamentos (shifts) e interpretação de valores binários.

MSB (Most Significant Bit)

Refere-se ao **bit mais significativo** dentro de uma sequência binária, é o bit mais à esquerda. Representa o **maior peso** (maior potência de 2). Ele determina a maior parcela do valor numérico representado e, em sistemas com sinal, pode ser usado como o bit que indica se o número é positivo ou negativo.

MinTermos (MinTerm)

São expressões que representam **uma única combinação específica** das entradas que torna a função igual a **1**. Cada mintermo corresponde a uma linha da tabela verdade onde a saída é verdadeira.

Esse método faz parte da **forma canônica Soma-de-Produtos (SOP)**, derivada dos estudos de **George Boole**, e sistematizada posteriormente no trabalho de **Claude Shannon**, que aplicou álgebra booleana diretamente a circuitos elétricos.

MaxTermos (MaxTerm)

Também são expressões que representam **uma única combinação específica** das entradas, porém, que torna a função igual a **0**. Cada maxtermo corresponde a uma linha da tabela verdade onde a saída é falsa.

O método segue a lógica complementar aos mintermos, integrante da **forma canônica Produto-de-Somas (POS)**, também fundamentada nos trabalhos iniciais de Boole e posteriormente organizada e aplicada à engenharia por **Shannon**.

Simplificação lógica

É o processo de reduzir uma expressão, função ou circuito lógico para uma forma equivalente **com menos termos, menos portas ou menor custo**. Muito utilizadas em MinTermos ou MaxTermos, utilizando técnicas como:

- Álgebra Booleana
- Mapa de Karnaugh

O objetivo é tornar o circuito **mais eficiente** — principalmente no mundo físico — mas mantendo o mesmo comportamento lógico.

Equivalência lógica

Duas expressões são logicamente equivalentes quando **produzem a mesma saída para todas as combinações de entrada**, mesmo que tenham formas diferentes.

Serve para validar simplificação, substituição de portas e otimizações. Por exemplo:

$$A \text{ OR } (A \text{ AND } B)$$

que é equivalente à apenas **A**.

1 — FUNDAMENTOS

1.1 — Glossário dos Fundamentos

1.1.1 — Termos de Circuitos

1.1.1.1 — Circuito combinacional

É um tipo de circuito digital cuja saída depende **exclusivamente dos valores atuais das entradas**, sem qualquer forma de armazenamento interno. Exemplos comuns incluem somadores, decodificadores, multiplexadores e portas lógicas. Sua operação é inteiramente determinada por expressões booleanas, portanto, para um mesmo conjunto de entradas, o circuito sempre produz a mesma saída.

1.1.1.2 — Circuito sequencial

Um circuito sequencial possui **memória interna**, o que significa que sua saída depende tanto das entradas atuais quanto do estado anterior, geralmente esses também dependem de clock. Esse estado é armazenado em elementos como flip-flops, shift registers (shifters) e registradores.

1.1.1.3 — Hierarquia de circuitos

É a organização dos componentes digitais em diferentes **níveis de abstração**, possibilitando compreender e projetar sistemas complexos de forma estruturada. Em uma hierarquia, circuitos simples formam módulos intermediários, que por sua vez compõem sistemas maiores.

1.1.1.4 — Bloco macro (Macro-Block ou apenas Macro)

Refere-se a um agrupamento funcional que **encapsula** um circuito mais complexo em um bloco simplificado. Um macro-block é uma forma de representação abstrata, permitindo que circuitos grandes sejam tratados como unidades únicas — como uma ULA, um multiplexador expandido ou qualquer função construída a partir de subcircuitos menores. Esses blocos possuem uma aparência característica, adotada como um consenso para a representação gráfica de macros em circuitos digitais.

Ao longo do desenvolvimento, eles serão apresentados tanto em formatos mais visuais — semelhantes a componentes físicos — quanto em versões customizadas ou que seguem padrões já consolidados. Em alguns casos, também serão utilizadas variações voltadas apenas para melhorar a estética e a organização dos circuitos.

1.1.1.5 — Modularidade de circuitos

Conceito que envolve dividir um circuito em partes independentes e reutilizáveis, chamadas **módulos**. Cada módulo executa uma função específica e pode ser projetado, testado e modificado de forma isolada.

1.1.1.6 — Componente / Subcircuito

Um subcircuito é um circuito menor encapsulado que realiza uma função específica e pode ser reutilizado dentro de um projeto maior. São peças menores, que tem uma função unitária.

1.1.1.7 — Componentes, Módulos e Macros

Componentes são blocos básicos do circuito, como portas lógicas, somadores de 1b bit, multiplexadores, displays, clocks e outros.

Já os módulos, são blocos funcionais formados por componentes conectados — como somador de 4 bits — que pode ser feito utilizando um método de conexão de 4 somadores de 1 bit, um dos mais conhecidos, é o método do **Ripple Carry**.

Os macros, são representações gráficas dos circuitos, mas que também servem como uma forma compactada e encapsulada de módulos e componentes, como somadores de 4 bits, para ser utilizado em uma ULA e nomeado como “Adder4”.

1.1.2 — Termos sobre as Operações

1.1.2.1 — Carry-in (Cin)

Valor de entrada que representa o “vai-um” **recebido** por um somador, normalmente proveniente de uma operação anterior ou de um bit menos significativo. Ele afeta diretamente o resultado da soma, principalmente em operações em cadeia, como no ripple carry.

1.1.2.2 — Carry-out (Cout)

Valor de saída que indica o “vai-um” **gerado** pela operação de soma do bit mais significativo daquele bloco. Em um somador de múltiplos bits, o Cout é propagado para o próximo estágio. Em operações aritméticas, também pode representar um estouro da capacidade daquele conjunto de bits.

1.1.2.3 — Overflow

Condição que ocorre quando o resultado de uma operação aritmética ultrapassa o limite de representação possível para aquele número de bits, especialmente em operações com números assinados. Ocorre quando o bit de sinal é calculado incorretamente devido ao resultado exceder o intervalo permitido.

1.1.2.4 — Borrow

Equivalente ao “empréstimo” utilizado em subtrações binárias. É um valor que representa que o bit menos significativo precisou “pedir” 1 ao bit seguinte para realizar a operação. Assim como o carry em somadores, o borrow se propaga entre bits em operações de subtração, e também há a existência do **borrow in** (Bin) e do **borrow out** (Bout), e seguem a mesma idéia de receber o empréstimo (Bin), e o emprestar (Bout).

1.1.2.5 — Ripple Carry

É o método de implementação de somadores em que o carry gerado por um bit é propagado sequencialmente ao próximo. O nome “ripple” descreve essa propagação gradual, que introduz atraso cumulativo, mas não caracteriza um circuito sequencial.

Também existe o **Ripple Borrow**, para circuitos de subtração, porém, da forma em que o subtrator será construído neste projeto, ele não será visto, apenas citado.

1.1.2.6 — Operação aritmética e lógica

Operações aritméticas abrangem cálculos como soma, subtração, divisão, até paridade (Par ou Ímpar) e primalidade (Números Primos), enquanto as operações lógicas envolvem manipulação bit a bit, como AND, OR, XOR e NOT. Ambas compartilham a mesma estrutura geral, mas têm finalidades distintas.

1.1.2.7 — Modo de Operação

Refere-se à configuração que define qual operação específica a ULA deve executar, geralmente selecionada por bits de controle. Cada modo habilita caminhos internos distintos, ativando a operação correspondente (ex.: modo soma, modo AND, modo OR).

1.1.2.8 — Circuitos “Full” e “Half”

Designações usadas para diferenciar se o circuito considera ou não um carry/borrow externo.

- Um **Half-Adder** (meio somador) calcula a soma de dois bits, sem carry-in.
- Um **Full-Adder** (somador completo) considera o carry-in, permitindo sua utilização em cadeias.

O mesmo raciocínio vale para subtratores, porém, não será usado por conta do método utilizado para construí-lo.

1.1.2.9 — Flags

Sinais auxiliares emitidos pela ULA que indicam condições específicas do resultado, como negativo, carry, overflow e outros. São utilizados por sistemas maiores para decidir ações posteriores. Neste projeto, apesar dos macros terem suporte para expansão, conseqüentemente para sinais, eles não serão utilizados.

1.1.3 — Termos relacionados à Simulação

1.1.3.1 — Simulação digital

Processo de modelar o comportamento de circuitos digitais em software, permitindo observar seu funcionamento sem necessidade de montar fisicamente o circuito.

Ferramentas como o Logisim reproduzem níveis lógicos, propagação de sinais e atrasos básicos. E o mesmo, será o software usado para desenvolver.

1.1.3.2 — Wire

Representa o fio que conecta componentes dentro do circuito. No simulador, é responsável por conduzir os valores lógicos entre entradas, saídas e blocos internos.

1.1.3.3 — Pin

Elemento de entrada ou saída usado para interagir com o circuito. Permite definir valores externos (entradas) ou observar resultados (saídas) durante a simulação.

A ferramenta utilizada permite que pins possam transmitir e receber diversos tamanhos de dados, desde apenas 1 bit, até maiores. Neste projeto, será apresentado uso de até 4 pins com 1 bit cada, até 1 pin com 4 bits, que podem ser divididos usando splitters, se necessário.

Eles não mudam a forma em que os dados são transmitidos, e permitem o uso de componentes já prontos para melhor estética e apresentação, como display hexadecimal.

1.1.3.4 — Splitter

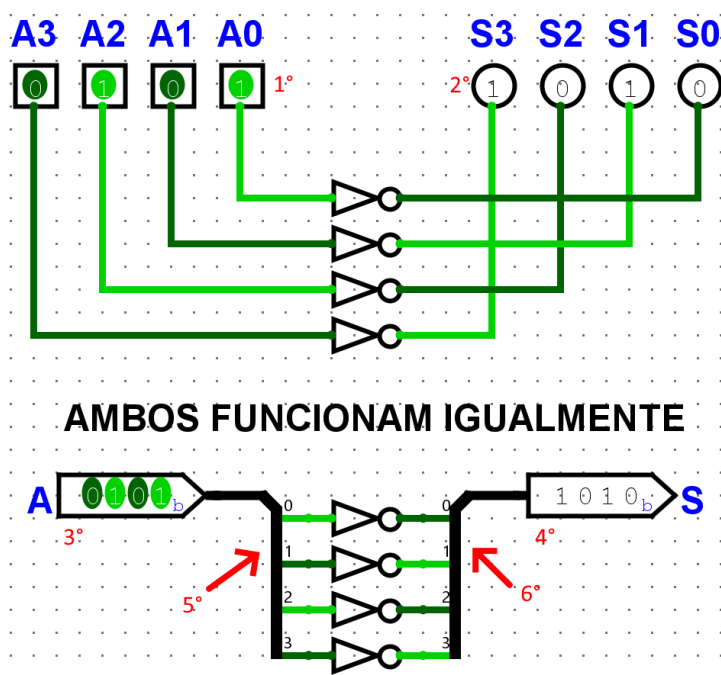
Componente que divide ou agrupa conjuntos de bits, permitindo manipular barramentos mais largos ou extrair partes específicas de uma sequência binária.

Ele consegue preservar a linha de dados de uma entrada e saída de dados, como exemplo: um pin de 4 bits de dados, consegue ser dividido em quatro linhas com um bit cada, e, em exemplo: caso apenas o quarto bit desse pin esteja em 1, apenas a quarta linha do splitter vai transmitir dados.

Para deixar mais claro, a Figura 1 apresenta a mesma lógica, de um circuito de uma porta lógica de 4 bits, construída usando quatro portas lógicas de 1 bit.

Ambos possuem o mesmo funcionamento, porém, a mais acima com 4 entradas de 1 bit precisam receber e enviar quatro linhas de dados diferentes, enquanto a mais abaixo, com apenas uma entrada de 4 bits, internamente consegue usar o splitter para dividir esses 4 bits em quatro linhas de dados diferentes, e depois, consegue fazer a junção dessas quatro linhas em uma saída de 4 bits.

Figura 1 — Exemplo de um circuito sem o uso e com o uso de pin e splitter



- 1: Pino de 1 bit de entrada com aparência clássica do Logisim;
- 2: Pino de 1 bit de saída com aparência clássica do Logisim;
- 3: Pino de 4 bits de entrada com aparência “Arrow”;
- 4: Pino de 4 bits de saída com aparência “Arrow”;
- 5: Splitter que consegue receber dados pela porta mais inclinada para esquerda, e dividir para outras quatro portas na parte vertical reta;
- 6: Splitter que consegue receber dados pelas portas na parte vertical, e juntá-los para uma única saída na porta mais inclinada para direita.

Percebe-se que, os splitters, no programa usado, não possuem diferenciadores muito visuais, além da direção e aparência, como “Right-Handed, East” e “Left-Handed, West”. Eles buscam respeitar os seus fios conectores, ele tenta “entender” de qual direção vem os dados, como pin de entrada e saída.

1.1.3.5 — Display Hex

Elemento visual que exibe um valor binário como um dígito ou grupo de dígitos em hexadecimal. Facilita a leitura de resultados em operações de diferentes bits.

1.1.3.6 — Clock

Sinal periódico usado para sincronizar circuitos sequenciais. Marca os instantes em que as operações devem mudar de estado. Em simuladores, pode ser configurado para pulsar automaticamente ou manualmente.

Para o projeto da ULA, esse não se mostra necessário, porém, será apresentado em outro trabalho.

1.1.3.7 — Label

Texto ou nome usado para identificar fios, blocos, entradas e saídas. Serve exclusivamente para organização, leitura e documentação do circuito.

1.1.3.8 — Tunnel

Recurso que permite conectar partes do circuito sem utilizar fios longos. Dois túneis com o mesmo nome funcionam como se estivessem diretamente ligados, facilitando a organização visual.

Eles serão utilizados apenas em momentos para representar algo de forma mais clara, pois, o seu uso intenso, apesar de funcional para o programa, pode causar confusão para o entendimento dos circuitos.

1.2 — Cálculos e Conversões

1.2.1 — Dígitos binários

Para determinar o valor representado por uma sequência binária, utiliza-se uma abordagem baseada nos pesos de cada posição. Essa técnica, bastante adequada para cálculos manuais, consiste em multiplicar cada dígito (bit) pelo peso correspondente à sua posição.

Como exemplo o número 1011_2 , temos:

$$1 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1 = 11$$

Nessa representação, o multiplicando é o dígito binário (0 ou 1), enquanto o multiplicador corresponde ao peso daquela casa, baseado nas potências de 2. A soma de todos os produtos resulta no valor decimal equivalente.

1.2.2 — Agrupamento em Nibbles

Para números binários maiores, é possível facilitar os cálculos utilizando agrupamentos de 4 bits (nibbles).

Por exemplo, a sequência $1011\ 1011_2$ pode ser dividida em dois grupos:

- primeiro nibble: 1011_2 ;
- segundo nibble: 1011_2

$$1 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1 = 11$$

No entanto, como cada nibble à esquerda representa valores 16 vezes maiores (pois ocupa as posições de 2^4 até 2^7), basta multiplicar o resultado do nibble mais significativo por 16:

$$16 \times 11 = 176$$

Somando com o valor do nibble menos significativo:

$$176 + 11 = 187$$

Ou seja:

$$1011\ 1011_2 = 187_{10}$$

1.2.3 — Por que multiplicar por 16

Um nibble possui valores possíveis de 0 a 15.

O próximo valor, 16, só pode ser representado com 5 bits — ou seja, um novo nibble inicia. Esse limite é o que chamamos, de forma informal, de “**teto + 1**”:

$$15 + 1 = 16$$

Por isso, cada nibble à esquerda desloca o valor em potências de 16, exatamente da mesma forma que, no sistema decimal, um dígito à esquerda representa dezenas, centenas, milhares etc.

Utilizando o número $0001\ 1111\ 1111_2$, temos três nibbles:

- $0001_2 = 1$
- $1111_2 = 15$
- $1111_2 = 15$

Os pesos para cada nibble são:

- $16^2 = 256$ para o primeiro
- $16^1 = 16$ para o segundo
- $16^0 = 1$ para o terceiro

Logo:

$$(256 \times 1) + (16 \times 15) + (1 \times 15) = 511$$

Essa simplificação, evita escrever uma soma extensa, como esta>

$$0001\ 1111\ 1111_2 = (0 \times 2^{11}) + (0 \times 2^{10}) + \dots + (1 \times 2^1) + (1 \times 2^0) = 511_{10}$$

1.2.4 — Conversões no Sistema Hexadecimal

O processo de conversão no sistema hexadecimal segue a mesma lógica, porém utilizando potências de 16, e não de 2. Um número como FF_{16} representa:

$$F = 15$$

$$(16 \times 15) + 15 = 255$$

É comum agrupar representações hexadecimais em pares de dígitos, pois dois dígitos hexadecimais equivalem a um byte (8 bits).

Também é prática recomendada deixar explícita a base utilizada, utilizando subscritos ou sufixos como:

- FF_{16}
- $0xFF$
- FFh

1.3 — Unidade Lógica Aritmética (ULA)

A Unidade Lógica e Aritmética (ULA) é um dos principais componentes de uma Unidade Central de Processamento (CPU), responsável por executar operações aritméticas e lógicas de forma direta e controlada. Dentro de um processador, a ULA atua como o bloco funcional onde cálculos são efetuados e decisões lógicas são tomadas, servindo como o ponto central de manipulação dos dados.

Em seu funcionamento, a ULA opera seguindo uma ordem definida de operações, que depende da forma como seu circuito interno é estruturado e do sinal de controle recebido. Essa ordem que a ALU executa as operações, depende da forma em que foi projetada, geralmente, essa ordem busca seguir um padrão, indo pela ordem alfabética dos nomes das operações em inglês, neste caso, a ordem:

- Primeira operação: Adder (Somador);
- Segunda operação: Buffer;
- Terceira operação: Nand (Não E);
- Quarta operação: Subtractor (Subtrator).

Essa ordem, é uma forma de padronizar o funcionamento da ULA, de forma que de forma simbólica, a primeira operação da ULA de um processador, sempre seja a mesma em todos os lugares onde for utilizada, assim evitando quebrar o seu funcionamento e causar problemas em situações reais.

A ULA construída possui duas entradas, InputA e InputB (ou simplesmente A e B), ambas com a mesma largura de bits da ULA. Nas operações aritméticas, cada entrada pode assumir valores entre 0 e 15. O resultado exibido também fica limitado a esse intervalo. Na subtração, o circuito não suporta situações em que B é maior que A, pois não há tratamento para resultados negativos nem indicação de sinal.

Da mesma forma, a operação de Buffer utiliza apenas a entrada A. Para fins de demonstração, ela simplesmente replica o valor de A na saída, enquanto a entrada B é ignorada.

Cada operação é implementada como um circuito específico, e a integração desses circuitos é feita por meio de multiplexadores, que selecionam qual resultado será disponibilizado ao restante da CPU.

Essa organização modular permite que a ULA seja expandida com novas operações, otimizando o desempenho e possibilitando sua adaptação a diferentes arquiteturas de processadores, desde sistemas simples até unidades mais complexas.

1.4 — Fundamentos dos Circuitos Lógicos Combinacionais Digital

1.4.1 — Portas lógicas

Os circuitos lógicos combinacionais são a base da computação digital, sendo responsáveis por produzir saídas que dependem exclusivamente das entradas atuais, sem armazenamento de estado.

Esses circuitos são formados pela combinação de portas lógicas fundamentais, que realizam operações elementares sobre sinais binários.

Em simulações digitais e em expressões simplificadas, uma porta lógica pode ter inúmeras entradas, mas, fisicamente elas são limitadas à forma com que são apresentadas. É comum ver portas lógicas de quatro entradas quando — por exemplo: a uma representação de um MinTermo — pois se trata de um circuito digital, mas, com componentes físicos, é raro encontrar, sendo necessário fazer conexões entre portas iguais, para resultar em uma saída.

1.4.1.1 — Vida real (CIs)

Em aplicações reais, as portas lógicas não existem isoladamente como representações abstratas, mas sim integradas em circuitos integrados (CIs) específicos. Cada CI possui um conjunto definido de portas, número de entradas, características elétricas e limitações próprias. Essas informações variam conforme o fabricante e o modelo do componente.

Esses circuitos são identificados por códigos padronizados — como a família **74HC**, **74LS**, **CD4000**, entre outras — que determinam tanto o tipo de porta contida quanto suas propriedades tecnológicas. Por exemplo, o CI **MM74HC32M** contém quatro portas OR de duas entradas.

O funcionamento elétrico, o número de portas disponíveis e a pinagem são especificados em seus respectivos *datasheets*, fornecidos pelos fabricantes. Esses *datasheets* são a fonte oficial de consulta e descrevem características como tensões de operação, tempos de comutação, diagramas internos e tabelas verdade.

Datasheets disponíveis em: <https://www.alldatasheet.com>

1.4.1.2 — Buffer

Embora o buffer não seja classificado como uma porta lógica fundamental, ele é amplamente utilizado em circuitos digitais, incluindo ULAs. Mas, seu principal uso está em circuitos físicos, onde há perda de força na carga elétrica, e outras funções do, como por exemplo:

- **reforço** (aumenta a capacidade de corrente para dirigir cargas maiores);
- **direcionar** energia para direções e específicas;
- **isolamento** entre estágios do circuito;
- **redução de ruído**;
- **proteção** contra interferências;
- e **controle da direção de sinais**, no caso de buffers *tri-state*.

Digitalmente, o buffer é um dispositivo cuja função básica é **replicar o sinal de entrada na saída**, sem alterá-lo. Em termos lógicos: **Saída = Entrada**.

Figura 2 — Símbolo da porta Buffer



Tabela 1 — Variações do buffer com entradas A (1 entrada)

A	S = A
0	0
1	1

1.4.1.3 — Porta NOT

A porta NOT realiza a operação de negação lógica. Ela possui apenas uma entrada e inverte seu valor: 0 torna-se 1, e 1 torna-se 0.

É fundamental para operações de complemento e geração de sinais inversos.

Figura 3 — Símbolo da porta lógica NOT



Tabela 2 — Variações da porta lógica NOT com entradas A (1 entrada)

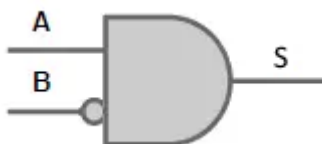
A	$\bar{A}$
0	1
1	0

Em raros casos, NOTs podem ser incluídas antes da conexão de uma entrada em portas lógicas.

Na Figura 4, é um exemplo de uma portá AND, porém, na entrada B, uma negação em seu sinal. Isso é o equivalente a inverter o sinal usando uma NOT, antes de conectar na entrada de portas lógicas.

Apesar de poder ser encontrado em diagramas, eles podem causar confusão para leitura, pois em em circuitos reais, são usados em casos onde, em um módulo só há a inversão de tal sinal, apenas uma vez, assim economizando na construção. Mas, digitalmente não faz tanta diferença, por conta disso, será evitado durante o projeto.

Figura 4 — Símbolo da porta lógica AND com uma NOT antes da conexão



1.4.1.4 — Porta AND

A porta AND (conjunção lógica) realiza uma operação em que a saída é igual a 1 somente quando **todas** as entradas também forem 1. Em qualquer outro caso, a saída será 0.

Usada quando é necessário garantir que duas condições ocorram simultaneamente.

Figura 5 — Símbolo da porta lógica AND

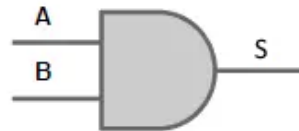


Tabela 3 — Variações da porta lógica AND com entradas A e B (2 entradas)

A	B	$S = A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1

1.4.1.5 — Porta OR

A porta OR (disjunção lógica) produz saída igual a 1 sempre que **ao menos uma** das entradas for 1. A saída será 0 apenas quando todas as entradas forem 0.

Usada para representar condições alternativas.

Figura 6 — Símbolo da porta lógica OR

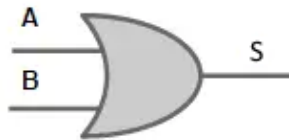


Tabela 4 — Variações da porta lógica OR com entradas A e B (2 entradas)

A	B	$S = A + B$
0	0	0
0	1	1
1	0	1
1	1	1

1.4.1.6 — Porta XOR

A porta XOR (OU exclusivo) produz saída igual a 1 quando **as entradas são diferentes** entre si.

Pode ser referenciada como uma “**detectora de diferenças**”, onde, caso as entradas sejam diferentes, a saída é 1.

É amplamente usada em circuitos aritméticos, especialmente em somadores, por sua capacidade de detectar diferenças entre bits.

Figura 7 — Símbolo da porta lógica XOR



Tabela 5 — Variações da porta lógica XOR com entradas A e B (2 entradas)

A	B	$S = A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

1.4.1.7 — Porta NAND

A porta NAND é a negação da operação AND. Assim, a saída é 0 somente quando todas as entradas forem 1; em qualquer outra combinação, a saída será 1.

Muito conhecido por ser uma **porta universal**, permitindo construir qualquer circuito lógico apenas com NANDs.

Figura 8 — Símbolo da porta lógica NAND

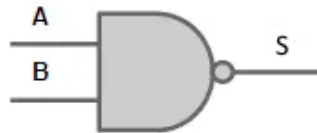


Tabela 6 — Variações da porta lógica NAND com entradas A e B (2 entradas)

A	B	$S = \overline{A \cdot B}$
0	0	1
0	1	1
1	0	1
1	1	0

1.4.1.8 — Porta NOR

A porta NOR é a negação da operação OR. A saída será 1 apenas quando todas as entradas forem 0; nos demais casos, será 0.

Assim como a NAND, também é considerada uma porta universal, permitindo a implementação de qualquer outro tipo de porta.

Figura 9 — Símbolo da porta lógica NOR

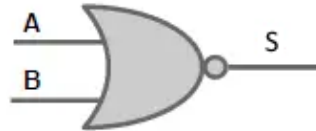


Tabela 7 — Variações da porta lógica NOR com entradas A e B (2 entradas)

A	B	$S = \overline{A + B}$
0	0	1
0	1	0
1	0	0
1	1	0

1.4.1.9 — Porta XNOR

A porta XNOR (OU exclusivo negado) é a negação da XOR. Sua saída é 1 quando **as entradas são iguais**.

Pode ser referenciada como uma “**detectora de igualdade**”, onde, caso as entradas sejam iguais, a saída é 1.

É útil em comparadores de igualdade e em determinadas técnicas de verificação de paridade.

Figura 10 — Símbolo da porta lógica XNOR



Tabela 8 — Variações da porta lógica XNOR com entradas A e B (2 entradas)

A	B	$S = \overline{A \oplus B}$
0	0	0
0	1	1
1	0	1
1	1	1

1.4.1.10 — Resumo dos símbolos em expressões

Tabela 9 — Diferentes representações dos símbolos das portas lógicas

Operação	Matemático	Álgebra
Buffer	A	$S = A$
NOT	$\neg A$	$\overline{A}$ ou A'
AND	$\wedge$	$A \cdot B$ ou $A . B$ ou AB
OR	$\vee$	$A + B$
XOR	$\oplus$	$A \oplus B$
NAND	$\uparrow$	$\overline{A \cdot B}$
NOR	$\downarrow$	$\overline{A + B}$
XNOR	$\leftrightarrow$	$\overline{A \oplus B}$ ou $A \odot B$

Todas as imagens dos símbolo lógicos foram retiradas de:

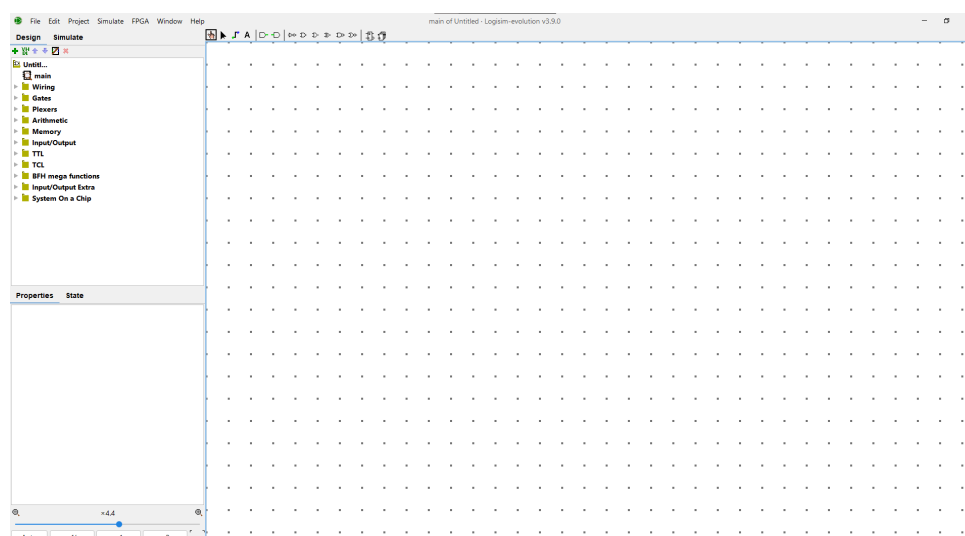
<https://embarcados.com.br/portas-logicas>

Elas foram recortadas, e outras, editadas trocando os símbolos de saída para S para melhor didática. A porta lógica XNOR foi criada com base na porta XOR existente, e a representação do Buffer também foi criada com base na porta NOT.

1.5 — Ferramenta de Simulação: Logisim

O Logisim-Evolution é um software open source para simulação de circuitos digitais, desenvolvido por colaboradores independentes e escrito em Java, o que garante compatibilidade com diferentes sistemas operacionais. Ele permite criar e testar circuitos de vários níveis de complexidade, desde portas lógicas básicas até ULAs e arquiteturas completas de processadores.

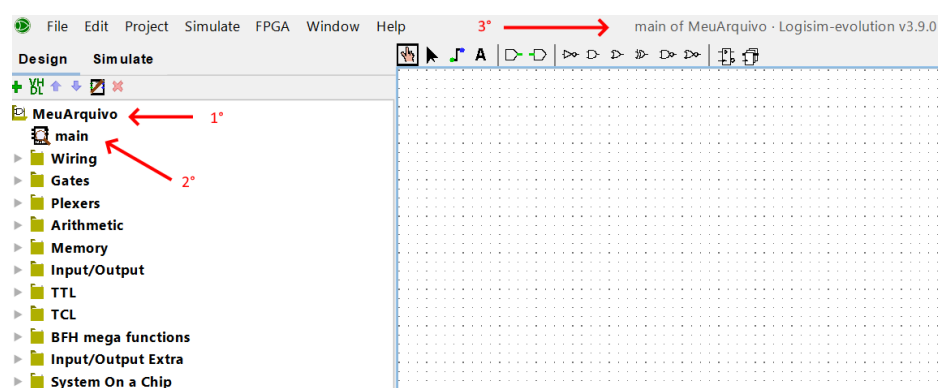
Figura 11 — Interface do Logisim-Evolution na versão v3.9.0



O programa já inclui diversas bibliotecas internas com portas lógicas, componentes aritméticos, multiplexadores, decodificadores e memórias. Esses elementos podem ter sua largura de bits ajustada nas configurações, permitindo que operem com vetores maiores sem a necessidade de replicar várias portas individuais de 1 bit, o que facilita a construção de circuitos modulares.

Cada projeto é salvo como um arquivo “.circ”, que contém tanto o circuito principal quanto seus subcircuitos. Esse formato funciona como uma biblioteca própria, podendo ser importado em outros projetos para reutilização de componentes já criados.

Figura 12 — Interface do Logisim-Evolution indicando o arquivo e seus circuitos



- O Logisim também trabalha em simulação contínua, mantendo o circuito sempre ativo e atualizando os resultados automaticamente conforme o usuário modifica valores ou componentes. É possível alternar entre o modo de edição e o modo de simulação para montar e testar o circuito de forma rápida e intuitiva.

The screenshot shows the Xilinx IDE interface. The top menu bar includes File, Edit, Project, Simulate, FPGA, Window, and Help. Below the menu bar, there are two tabs: Design and Simulate. The Design tab is active, and the Simulate tab is highlighted. The Design tab contains icons for Run, Stop, and other simulation controls. The Simulate tab contains icons for Run, Stop, and other simulation controls. Red arrows and degree labels (1° to 7°) indicate the sequence of actions for running a simulation.

- A ferramenta, já consta com funções internas de análise lógica, que permitem — por exemplo — uma análise lógica do circuito, para gerar tabelas verdades, expressões booleanas e até mapas de otimização. Além de, quando necessário, o programa consegue gerar um circuito completo a partir das variáveis e tabela verdade, e até expressões booleanas. A geração automática, não será usada para o desenvolvimento, quando necessário, a função para transformar o circuito em tabela verdade e expressão booleana.

1.5.1 — Uso do Logisim

Anteriormente, o CircuitMaker chegou a ser considerado para o desenvolvimento do projeto, mas acabou sendo deixado de lado, após uma análise das necessidades práticas. Optou-se pelo Logisim por oferecer um ambiente mais estável, atualizado com maior frequência e mais adequado para simulações acadêmicas.

Além disso, o Logisim inclui funcionalidades que tornam o trabalho mais simples no dia a dia, como salvamento automático, personalização de cores, uma biblioteca de componentes mais ampla, suporte a vários idiomas e uma interface mais acessível.

Outro ponto relevante é o suporte eficiente a atalhos de teclado — como Ctrl+Z, Ctrl+C e Ctrl+S — que agilizam o uso do programa. A interface mais moderna também ajuda na organização visual dos circuitos, facilitando a navegação e a compreensão do projeto.

2 — MÉTODO DE DESENVOLVIMENTO

2.1 — Processo de Construção dos Circuitos

A construção da Unidade Lógica e Aritmética (ULA) de 4 bits seguiu um processo incremental, começando pelos circuitos mais simples e avançando gradualmente para os mais complexos.

O primeiro módulo desenvolvido foi o **Buffer** de 4 bits, criado por meio do encadeamento de quatro buffers de 1 bit. Em seguida, foi implementada a porta **NAND** de 4 bits, também formada pela replicação direta de quatro portas NAND individuais, cada uma recebendo os sinais correspondentes dos barramentos A e B.

Após esses módulos, iniciando pelo **Somador**, construiu-se um meio somador como base teórica para o funcionamento da operação de soma. Em seguida, foi criado o somador completo de 1 bit, combinando meio somador e portas lógicas adicionais. Com esse componente pronto, o somador de 4 bits foi formado pelo método de ripple carry, onde quatro somadores completos são conectados sequencialmente, propagando o carry de um estágio ao próximo.

Para a operação de **Subtração**, foi adotado o método do complemento de dois, realizando a operação $A + \text{complemento de } B$, o que permite reutilizar a mesma estrutura do somador para executar a subtração de forma eficiente.

Além das operações principais da ULA, também foram preparados o decodificador 2x4, responsável por ativar uma entre quatro linhas de controle com base em um código de 2 bits, e o multiplexador 4x1, que seleciona qual das operações será enviada à saída final.

Apesar da ferramenta, oferecer suporte para fios que transportam diversos bits, e entradas e saídas em pins que também oferecem suporte para mais bits — para melhor didática — será apresentado dois modelos da ULA. Uma priorizando o entendimento, deixando de forma mais visível o transporte de bits, como macros com diversas entradas e saídas, e outro priorizando a estética do circuito, estética essa que — para desenvolvimentos futuros, como a CPU simples, serão mais utilizados, melhorando o entendimento circuitos e disponibilizando uma forma mais clara dos componentes.

Inclusive a criação de Macros para os circuitos, também seguiram essa idéia, apresentando blocos mais padronizados e blocos com um visual mais agradável, alguns inclusive, seguindo os diagramas reais, como o design dos multiplexadores, decodificadores e da ULA.

2.2 — Testes e Validações do Projeto

A validação dos circuitos será realizada diretamente na interface do Logisim, permitindo testar cada módulo individualmente antes da integração final.

A validação seguirá um processo iterativo, garantindo o funcionamento correto de cada bloco antes que ele seja incorporado à ULA completa.

3 — DESENVOLVIMENTO DOS CIRCUITOS DA ULA

3.1 — Circuito: Buffer de 4 bits

Um circuito de um Buffer de 4 bits, tem a função de replicar o sinal de uma entrada na saída, respeitando cada linha de dados.

Como citado, a forma em que o Buffer de 4 bits será construído, será utilizado o método do encadeamento de quatro buffers de 1 bit.

3.1.1 — Expressão Matemática

$$S3 = A3$$

$$S2 = A2$$

$$S1 = A1$$

$$S0 = A0$$

3.1.2 — Tabela Verdade

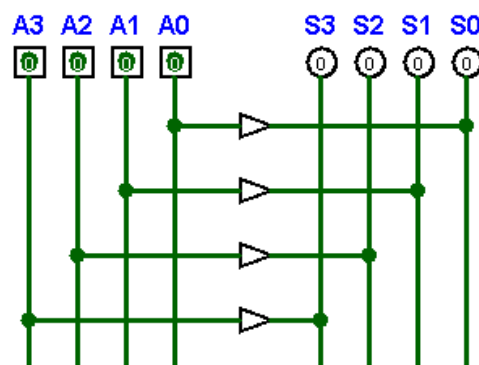
Como já explicado, a função do buffer é apenas replicar o sinal de entrada na saída, por conta disso, a tabela verdade foi minimizada.

Figura 14 — Tabela Verdade: Buffer 4 bits, minimizada

A3	A2	A1	A0	S3	S2	S1	S0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
		...			...		
1	1	1	0	1	1	1	0
1	1	1	1	1	1	1	1

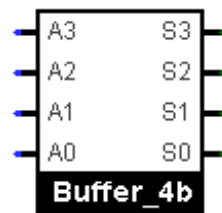
3.1.3 — Circuito Lógico Padrão

Figura 15 — Circuito Lógico Padrão: Buffer 4 bits



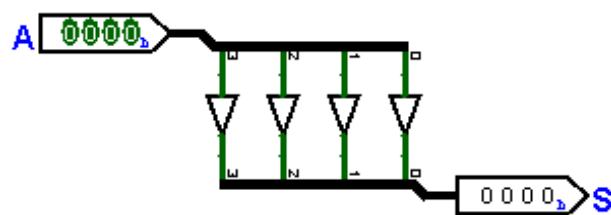
3.1.4 — Bloco Macro Padrão

Figura 16 — Bloco Macro Padrão: Buffer 4 bits



3.1.5 — Circuito Lógico Customizado

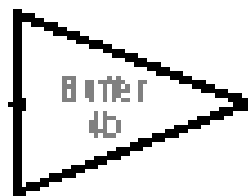
Figura 17 — Circuito Lógico Customizado: Buffer 4 bits



3.1.6 — Bloco Macro Customizado

A qualidade da imagem prejudicou o texto de identificação no Bloco Padrão, então, transcrevendo o texto: Buffer 4b

Figura 18 — Bloco Macro Customizado: Buffer 4 bits



3.2 — Circuito: Nand 4 bits (NE)

Um circuito NAND de 4 bits tem o objetivo de aplicar a operação lógica NAND bit a bit entre duas entradas, cada uma contendo 4 linhas de dados.

Para construir esse circuito de forma modular, utiliza-se o encadeamento de quatro portas NAND de 1 bit. Semelhante a construção do Buffer, porém, com duas entradas ao invés de uma.

Essa mesma lógica de encadeamento pode ser aplicada para expandir outras portas lógicas de 1 bit, como NOT, AND, OR, NOR, XOR e XNOR.

3.2.1 — Expressão Matemática

$$S3 = \overline{A3 \cdot B3}$$

$$S2 = \overline{A2 \cdot B2}$$

$$S1 = \overline{A1 \cdot B1}$$

$$S0 = \overline{A0 \cdot B0}$$

3.2.2 — Tabela Verdade/Funcional

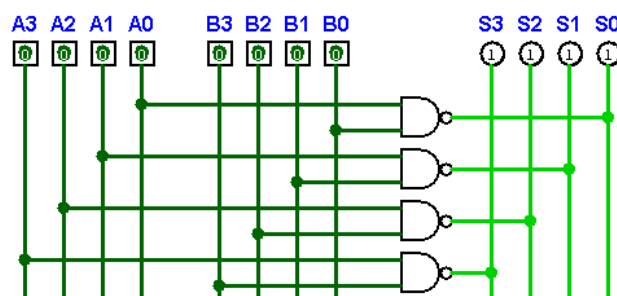
Esta tabela possui 256 linhas, então apresenta-se uma tabela funcional, onde “i” indicam “para cada bit”.

Figura 19 — Tabela Verdade: NAND 4 bits

A_i	B_i	$A_i \text{ NAND } B_i$
0	0	1
0	1	1
1	0	1
1	1	0

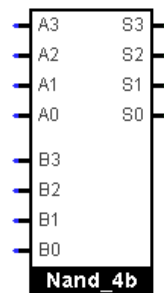
3.2.3 — Circuito Lógico Padrão

Figura 20 — Circuito Lógico Padrão: NAND 4 bits



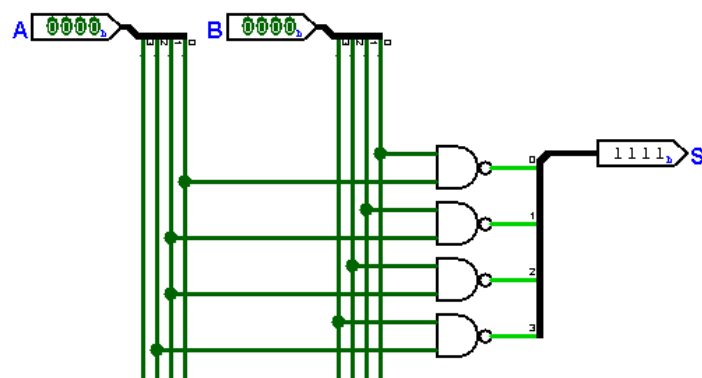
3.2.4 — Bloco Macro Padrão

Figura 21 — Bloco Macro Padrão: NAND 4 bits



3.2.5 — Circuito Lógico Customizado

Figura 22 — Circuito Lógico Customizado: NAND 4 bits



3.2.6 — Bloco Macro Customizado

Transcrição do texto: Nand 4b

Figura 23 — Bloco Macro Customizado: NAND 4 bits



3.3 — Circuito: Adder 4 bits (Somador)

A soma na lógica digital é construída a partir de operações básicas entre bits. Como cada bit só pode assumir o valor 0 ou 1, o processo de adicionar dois bits funciona de forma semelhante à adição na matemática convencional, mas utilizando portas lógicas para determinar tanto o bit de resultado quanto o transporte para a próxima posição. O comportamento dessa soma começa de forma simples, considerando apenas dois bits, e depois é expandido para lidar com entradas maiores.

O primeiro passo para entender essa construção é observar o funcionamento do somador mais simples, conhecido como **Half Adder**. Esse circuito recebe dois bits e produz dois resultados independentes: o bit de soma e o bit de transporte. O bit de soma indica o resultado direto da adição dos dois valores, enquanto o transporte representa o “vai-um” que ocorre quando ambos os bits são 1. Esse circuito não recebe nenhum transporte, e por possuir uma XOR e uma AND em seu circuito, também serve como otimização para a expressão de um somador completo.

Para lidar com situações onde existe um transporte anterior, é usado o **somador completo**, também referenciado como **Full Adder**. Esse novo circuito considera três entradas: os dois bits que estão sendo somados e o transporte vindo da posição anterior. Com isso, ele consegue gerar tanto o bit de resultado quanto um novo transporte, que será encaminhado para a próxima posição da soma.

Com o Full Adder definido, torna-se possível construir somadores maiores simplesmente encadeando vários desses blocos. Esse método é chamado de **ripple carry**, pois o transporte se propaga de um estágio para o outro, passando pelo circuito inteiro. Cada Full Adder é responsável por somar um par de bits correspondentes, levando em conta o transporte que recebeu. Após calcular seu próprio resultado, ele gera um novo transporte, que “rippleia” (se espalha) para o próximo somador na cadeia.

Essa estrutura modular permite a construção de um **somador de 4 bits** utilizando quatro somadores completos conectados em série. O primeiro estágio recebe o transporte inicial, normalmente definido como zero, e calcula a soma do bit menos significativo. Seu transporte passa para o próximo somador, que utiliza essa informação para realizar seu próprio cálculo. Esse processo continua de forma sequencial até o último estágio, que produz o transporte final da operação, caso exista. Assim como as portas lógicas de 1 bit podem ser encadeadas, o somador também segue essa lógica de expansão natural.

Essa mesma lógica de controle, pode ser usada para montar um subtrator, porém, há um método mais eficiente, que usa o complemento da segunda entrada.

3.3.1 — Adder Half (AdderH)

3.3.1.1 — Expressão Matemática

$$S = A \oplus B$$

$$Co = A \cdot B$$

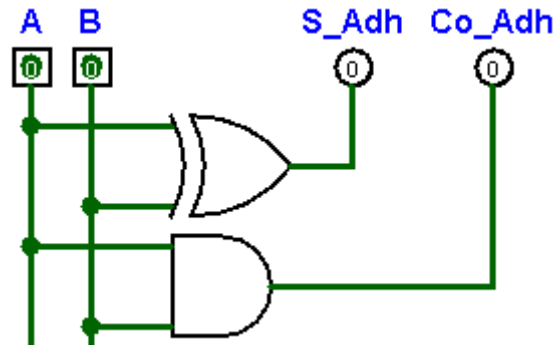
3.3.1.2 — Tabela Verdade

Figura 24 — Tabela Verdade: Adder Half

A	B	$S = A \oplus B$	$Co = A \cdot B$
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

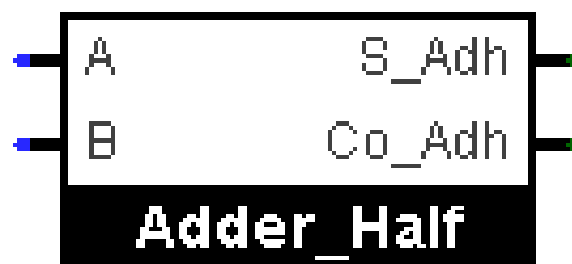
3.3.1.3 — Circuito Lógico Padrão

Figura 25 — Circuito Lógico Padrão: Adder Half



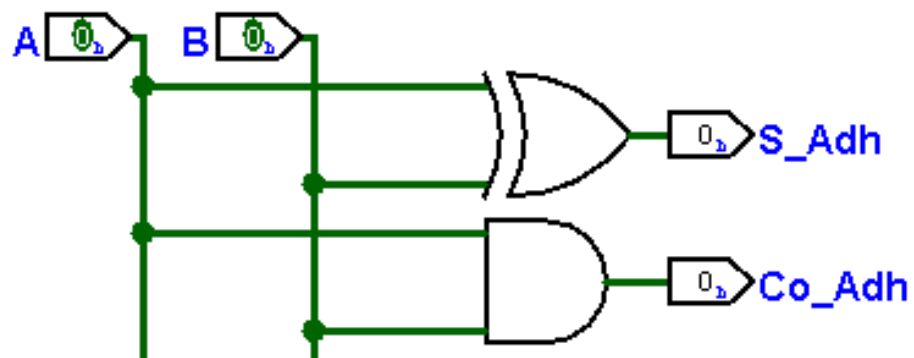
3.3.1.4 — Bloco Macro Padrão

Figura 26 — Bloco Macro Padrão: Adder Half



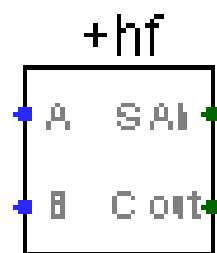
3.3.1.5 — Circuito Lógico Customizado

Figura 27 — Circuito Lógico Customizado: Adder Half



3.3.1.6 — Bloco Macro Customizado

Figura 28 — Bloco Macro Customizado: Adder Half



3.3.2 — Adder 1 bit (Adder1b)

$$(A \cdot B) + (Ci \cdot (A \oplus B))$$

3.3.2.1 — Expressão Matemática

$$S = A \oplus B \oplus Ci$$

$$Co = (A \cdot B) + (Ci \cdot (A \oplus B))$$

Essas expressões podem ser feitas de uma forma otimizada. Forma essa, que pode ser usada como base para montar um circuito mais eficiente:

$$T1 = A \oplus B$$

$$T2 = A \cdot B$$

$$T3 = Ci \cdot T1$$

$$S = T1 \oplus Ci$$

$$Co = T2 + T3$$

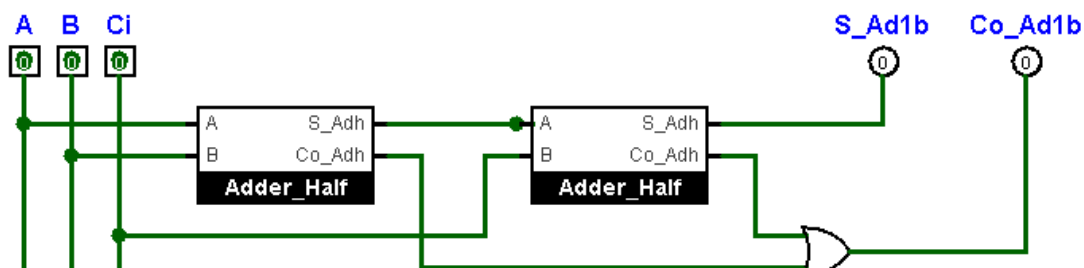
3.3.2.2 — Tabela Verdade

Figura 29 — Tabela Verdade: Adder 1 bit

A	B	Ci	S_Ad1b	Co_Ad1b
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

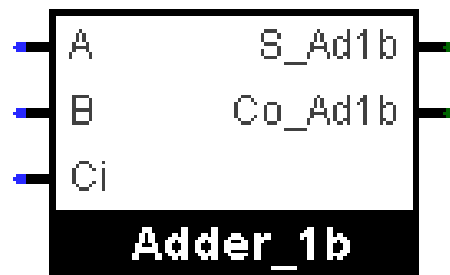
3.3.2.3 — Circuito Lógico Padrão

Figura 30 — Circuito Lógico Padrão: Adder 1 bit



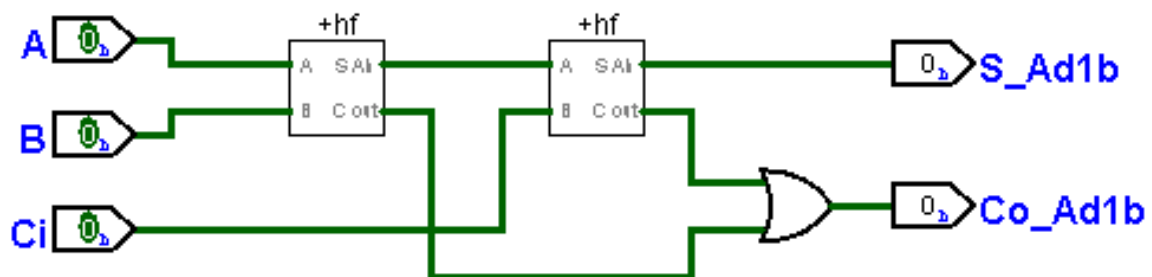
3.3.2.4 — Bloco Macro Padrão

Figura 31 — Bloco Macro Padrão: Adder 1 bit



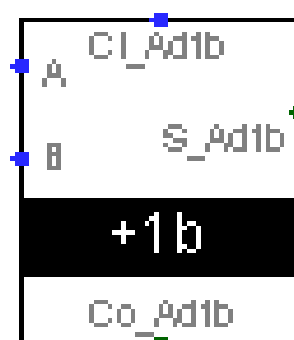
3.3.2.3 — Circuito Lógico Customizado

Figura 32 — Circuito Lógico Customizado: Adder 1 bit



3.3.2.4 — Bloco Macro Customizado

Figura 33 — Bloco Macro Customizado: Adder 1 bit



3.3.3 — Aplicando no Adder 4 bits (Ripple-Carry)

3.3.3.1 — Expressão Matemática.

O índice i representa a posição do bit, indo do menos significativo ($i = 0$) ao mais significativo ($i = 3$). Assim, A_i e B_i são os bits correspondentes das entradas A e B.

O transporte entre estágios é representado por C , onde C_i é o carry que chega ao bit i , e C_{i+1} é o carry que será enviado para o próximo estágio. Portanto, cada carry depende sempre do carry produzido no bit anterior.

$$S_i = A_i \oplus B_i \oplus C_i$$

$$C_{i+1} = (A_i \cdot B_i) + (C_i \cdot (A_i \oplus B_i))$$

E para saída da soma completa e do carry final:

$$S = (S_3 \cdot S_2 \cdot S_1 \cdot S_0)$$

$$Co = C_4$$

3.3.3.2 — Tabela Verdade

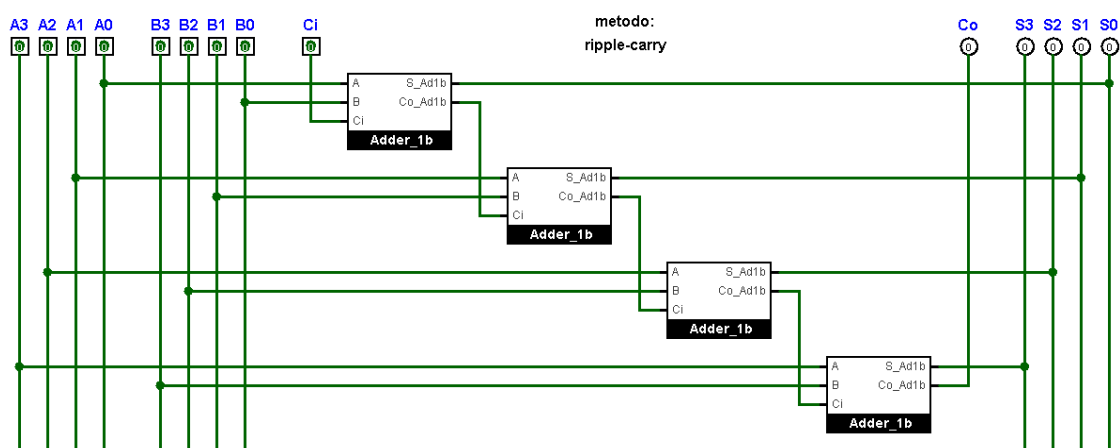
A tabela dessas operações geram variações gigantes, então apresenta-se um exemplo da tabela funcional, onde as linhas são os valores e os resultados.

Figura 34 — Tabela Verdade: Adder 4 bits

Entradas			Saídas	
A	B	Ci	S	Co
0101	0011	0	1000	0
1111	0001	0	0000	1
1010	0111	1	0010	1

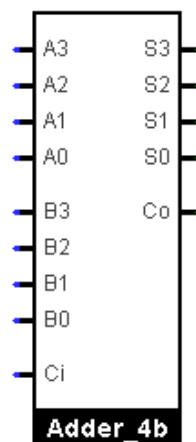
3.3.3.3 — Circuito Lógico Padrão

Figura 35 — Circuito Lógico Padrão: Adder 4 bits



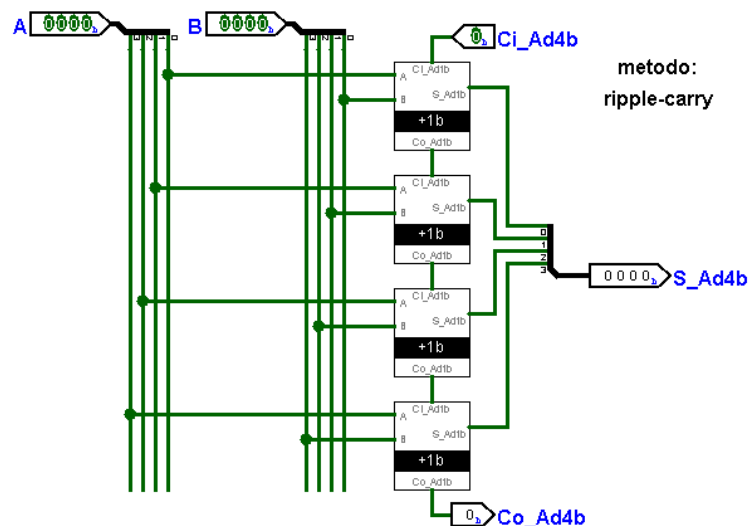
3.3.3.4 — Bloco Macro Padrão

Figura 36 — Bloco Macro Padrão: Adder 4 bits



3.3.3.5 — Circuito Lógico Customizado

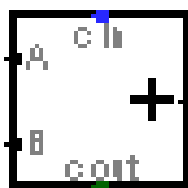
Figura 37 — Circuito Lógico Customizado: Adder 4 bits



3.3.3.6 — Bloco Macro Customizado

Transcrição: Parte superior “c in”, parte inferior “c out”

Figura 38 — Bloco Macro Customizado: Adder 4 bits



3.4 — Circuito: Subtractor 4 bits (Subtração)

A subtração em circuitos digitais segue um princípio semelhante ao da soma, porém utilizando uma transformação da entrada B. Como citado, o método adotado consiste em aplicar o complemento de dois em B, permitindo que a operação de subtração seja realizada como uma soma modificada. Para isso, a primeira etapa consiste em inverter todos os bits de B, algo facilmente obtido utilizando o circuito NAND de 4 bits já construído, configurado de modo a atuar como inversor para cada linha de dados.

Após a inversão, soma-se o valor 1 por meio do transporte inicial, completando a formação do complemento de dois. Com B transformado, o circuito passa a calcular $A - B$ utilizando o mesmo somador empregado na operação de adição, sem a necessidade de componentes dedicados exclusivamente à subtração. Essa abordagem se mostra mais eficiente e modular, depois de expandir a soma, basta reajustar os valores no subtrator, tratando dois circuitos de uma vez.

Assim como na soma de múltiplos bits, o transporte se propaga entre os estágios conforme o método ripple carry. A diferença está no fato de que o primeiro carry-in não é zero, mas sim 1, garantindo a finalização do processo de complemento. Esse valor propagado, em subtratores, é chamado de borrow in e borrow out.

O resultado produzido representa o valor subtraído, enquanto o carry final pode atuar como indicador de empréstimo (borrow) ou ser ignorado, também pode ser utilizado como sinal de saída, que ao ser tratado, pode servir como um indicador de um número negativo. Esse método unifica soma e subtração dentro da mesma estrutura lógica, simplificando a implementação da ULA.

3.4.1 — Expressão Matemática

A expressão é baseada no método utilizado, usando o complemento de B.

$$S = A - B = A + (\overline{B}) + 1$$

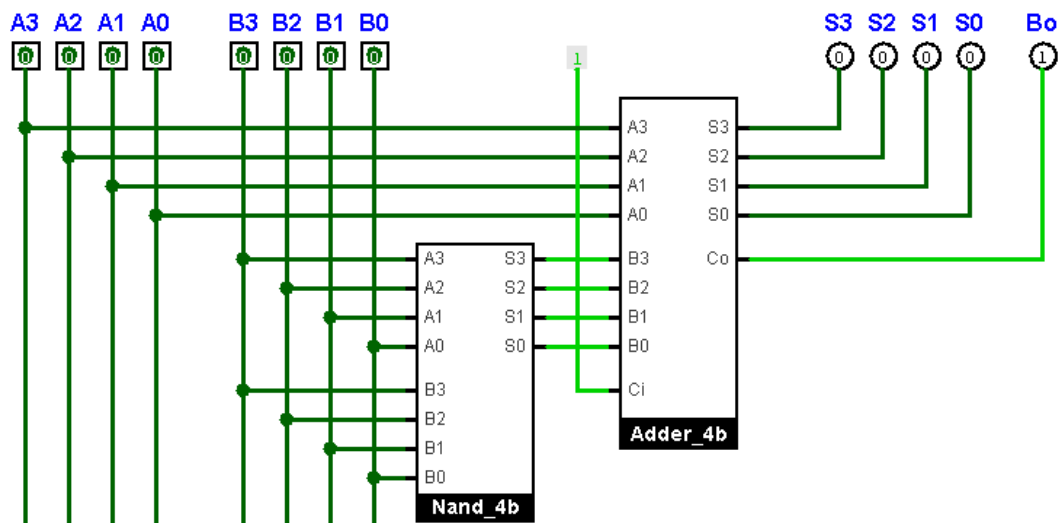
3.4.2 — Tabela Verdade

Figura 39 — Tabela Verdade: Subtractor 4 bits

Entradas			Saídas	
A	B	Bi	S	Bo
1001	0011	0	0110	0
0101	1010	0	1011	1
0110	0110	0	0000	0

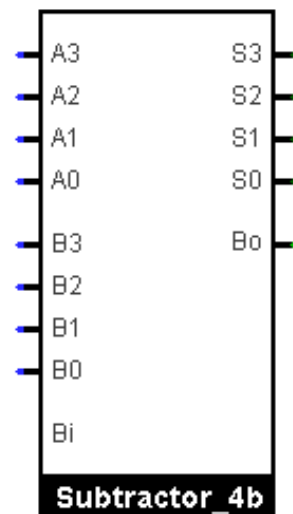
3.4.3 — Circuito Lógico Padrão

Figura 40 — Circuito Lógico Padrão: Subtractor 4 bits



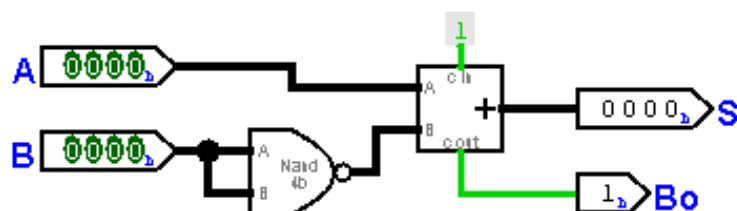
3.4.4 — Bloco Macro Padrão

Figura 41 — Bloco Macro Padrão: Subtractor 4 bits



3.4.5 — Circuito Lógico Customizado

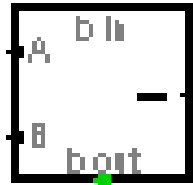
Figura 42 — Circuito Lógico Customizado: Subtractor 4 bits



3.4.6 — Bloco Macro Customizado

Transcrição: Parte superior “b in”, parte inferior “b out”

Figura 43 — Bloco Macro Customizado: Subtractor 4 bits



3.5 — Circuito: Decoder (Decodificador)

Os decodificadores são circuitos responsáveis por converter um conjunto reduzido de bits de seleção em múltiplas linhas de saída, onde exatamente uma delas é ativada. Eles funcionam como uma forma de “traduzir” o valor binário aplicado na entrada para a habilitação de uma única operação entre várias possíveis. Isso é especialmente útil quando o circuito precisa escolher apenas um caminho ou função a ser executada.

No contexto da ULA, cada nova operação adicionada aumenta o número total de funções possíveis, e conseqüentemente, o número de linhas que precisam ser selecionadas. Por isso, para cada multiplexador cujo conjunto de operações seja diferente ou maior do que o anterior, torna-se necessário criar um decoder compatível com a nova quantidade de seletores. Ele garante que somente a operação desejada seja habilitada, enquanto todas as outras permanecem desativadas.

3.5.1 — Expressão Matemática

Os “m” indicam a saída dos dados, que se forem 1, indicam a resposta positiva da tentativa de passagem, quando forem utilizados em outros momentos, como em multiplexadores.

$$m0 = \bar{A} \cdot \bar{B}$$

$$m1 = A \cdot \bar{B}$$

$$m2 = \bar{A} \cdot B$$

$$m3 = A \cdot B$$

3.5.2 — Tabela Verdade

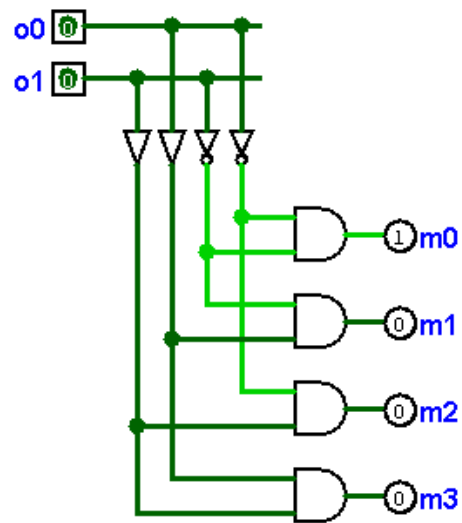
Os “OpX” indicam o bit referente ao input que escolhe a operação, estão nomeados dessa forma, pois é como serão usados durante a construção do circuito, mas, eles são equivalentes ao A e B usados anteriormente.

Figura 44 — Tabela Verdade: Decoder

Op1	Op0	m0	m1	m2	m3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

3.5.3 — Circuito Lógico Padrão

Figura 45 — Circuito Lógico Padrão: Decoder



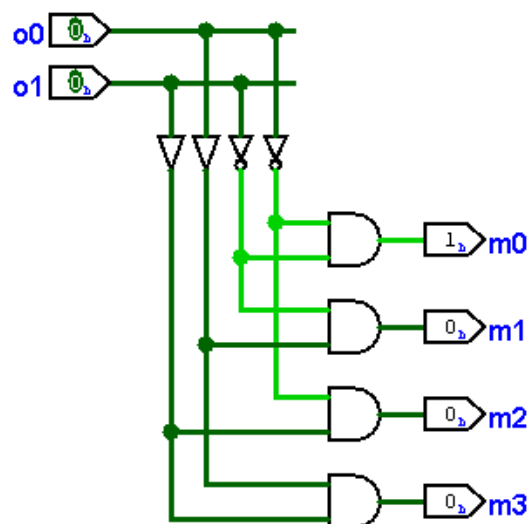
3.5.4 — Bloco Macro Padrão

Figura 46 — Bloco Macro Padrão: Decoder



3.5.5 — Circuito Lógico Customizado

Figura 47 — Circuito Lógico Customizado: Decoder



3.5.6 — Bloco Macro Customizado

Figura 48 — Bloco Macro Customizado: Decoder



3.6 — Circuito: Mux 4 bits (Multiplexer) (Multiplexador)

3.6.1 — Mux 1 bit (Mux1b)

Os multiplexadores são dispositivos responsáveis por selecionar um único sinal dentre várias entradas possíveis, encaminhando-o diretamente para a saída. Essa seleção é controlada por um conjunto de bits, conhecidos como linhas de seleção, que determinam qual das entradas estará ativa naquele momento. Dessa forma, o multiplexador atua como um “comutador lógico”, permitindo que diferentes fontes de dados compartilhem a mesma linha de saída sem interferência.

Dentro de uma ULA, os multiplexadores são usados para escolher o resultado final entre várias operações disponíveis, como soma, subtração e funções lógicas. Sempre que novas operações são adicionadas ao sistema, torna-se necessário ajustar ou expandir o multiplexador para comportar o número adequado de entradas. A seleção dessas entradas depende diretamente do decodificador correspondente, que gera os sinais de ativação corretos para cada função.

3.6.1.1 — Expressão Matemática

Nesta situação, a expressão não possui valores fixos, pois variam com o número de operações. Aqui temos quatro entradas “E”, dois bits de seleção e uma saída, todos com apenas 1 bit de dados.

$$\begin{aligned}S0 &= \overline{Op1} \cdot \overline{Op0} \cdot E0 \\S1 &= Op1 \cdot \overline{Op0} \cdot E1 \\S2 &= \overline{Op1} \cdot Op0 \cdot E2 \\S3 &= Op1 \cdot Op0 \cdot E3 \\S &= S0 + S1 + S2 + S3\end{aligned}$$

3.6.1.2 — Tabela Verdade

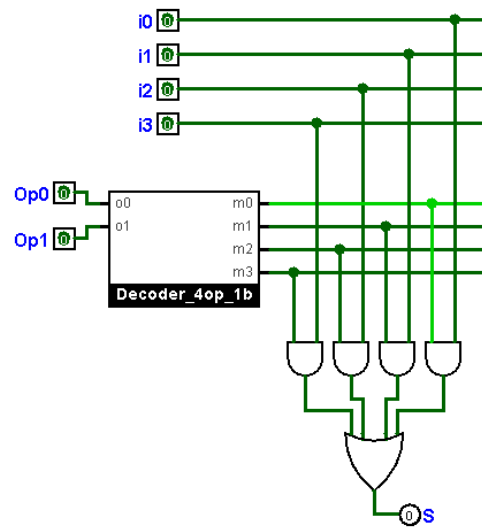
Tabela otimizada das operações. Levando em conta que, cada saída é apenas 1 bit.

Figura 49 — Tabela Verdade: MUX 1 bit

Op1	Op0	s4
0	0	i0
0	1	i1
1	0	i2
1	1	i3

3.6.1.3 — Circuito Lógico Padrão

Figura 50 — Circuito Lógico Padrão: MUX 1 bit



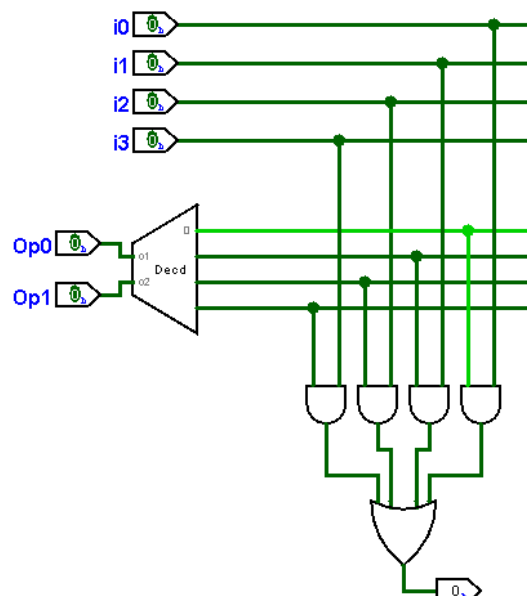
3.6.1.4 — Bloco Macro Padrão

Figura 51 — Bloco Macro Padrão: MUX 1 bit



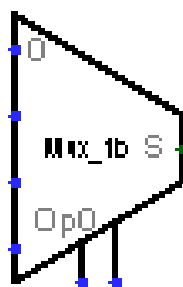
3.6.1.5 — Circuito Lógico Customizado

Figura 52 — Circuito Lógico Customizado: MUX 1 bit



3.6.1.6 — Bloco Macro Customizado

Figura 53 — Bloco Macro Customizado: MUX 1 bit



3.6.2 — Aplicando no MUX 4 bits (Encadeamento)

Assim como outros circuitos digitais modulares, os multiplexadores também podem ser expandidos por encadeamento. Para construir um multiplexador de 4 bits, utiliza-se quatro multiplexadores de 1 bit, todos compartilhando as mesmas linhas de seleção. Cada multiplexador individual controla apenas um bit da operação, e, ao serem combinados, formam um único multiplexador de diversos bits capaz de selecionar simultaneamente os quatro bits de uma operação completa.

3.6.2.1 — Expressão Matemática

A expressão do multiplexador de 4 bits, segue a mesma base do Mux de 1 bit, porém, levando em conta que os dados transportados para a saída e recebidos são de 4 bits, com exceção do seletor das operações que continuam 2 bits, e tratados de forma individual por cada multiplexador interno.

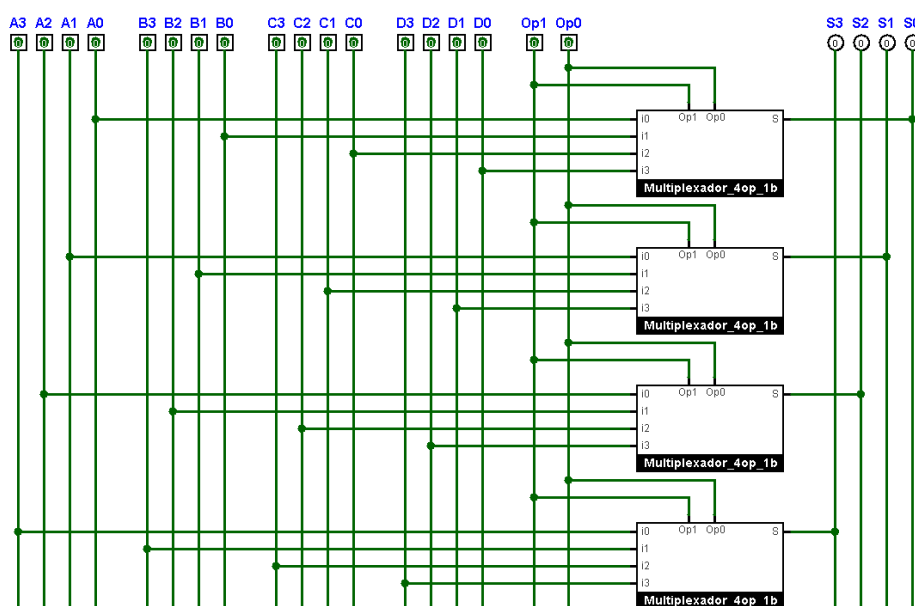
3.6.2.2 — Tabela Verdade

Figura 54 — Tabela Verdade: MUX 4 bits

Op	S
00	Input0
01	Input1
10	Input2
11	Input3

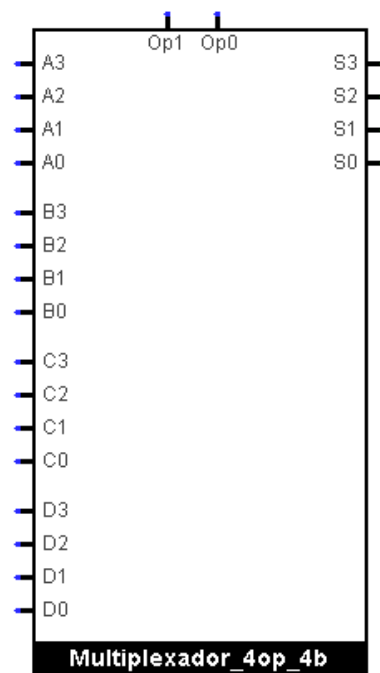
3.6.2.3 — Circuito Lógico Padrão

Figura 55 — Circuito Lógico Padrão: MUX 4 bits



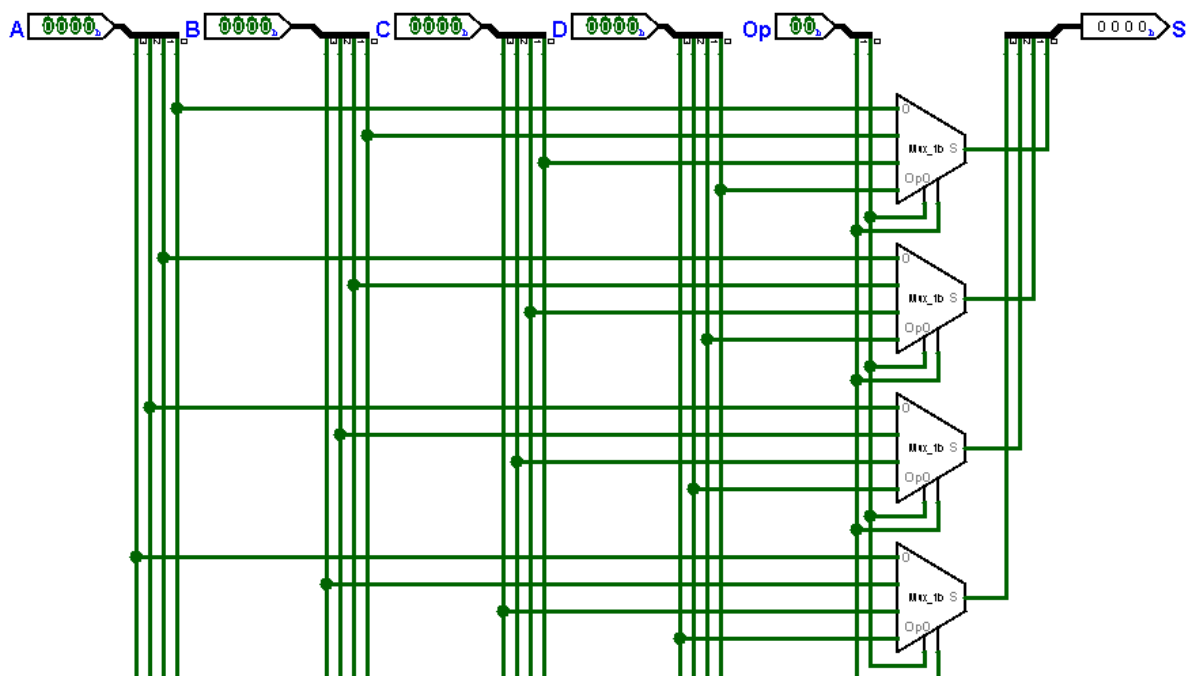
3.6.2.4 — Bloco Macro Padrão

Figura 56 — Bloco Macro Padrão: MUX 4 bits



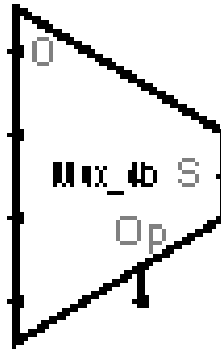
3.6.2.5 — Circuito Lógico Customizado

Figura 57 — Circuito Lógico Customizado: MUX 4 bits



3.6.2.6 — Bloco Macro Customizado

Figura 58 — Bloco Macro Customizado: MUX 4 bits



4 — MONTAGEM DA ULA

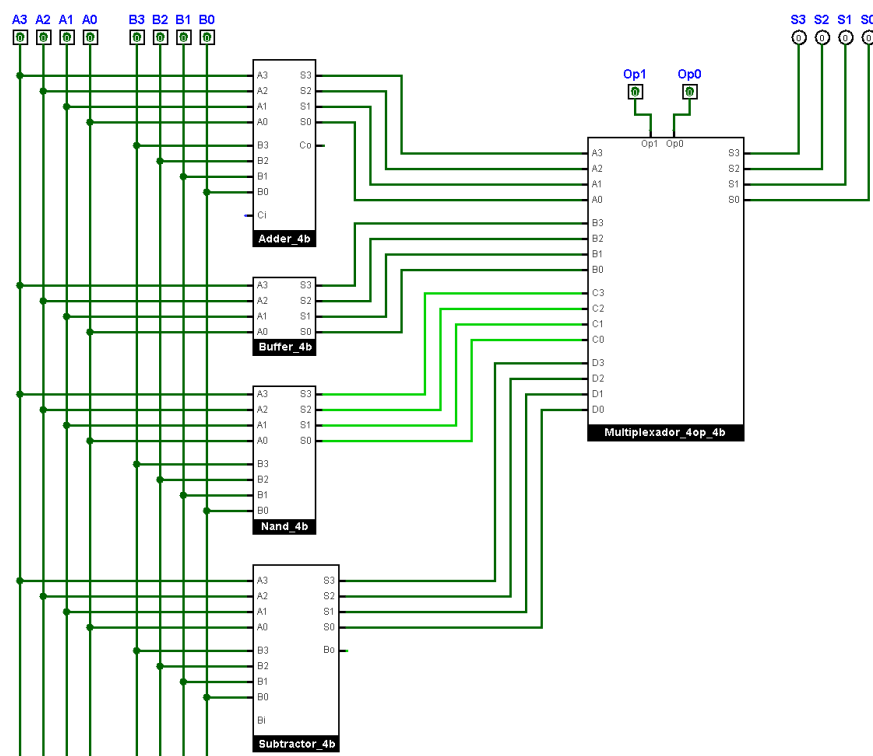
Finalmente, para concluir a construção da ULA de 4 bits e 4 operações, basta realizar as conexões entre os Barramentos de entrada “A” e “B”, internamente, conectá-los da forma esperada em cada um das operações, a saída das operações, conectá-las nas entradas do Mux de 4 bits criado, respeitando a ordem já mencionada, em seguida, utilizar duas entradas para escolher as operações conectando-as no Mux, e finalizar com ligando as saídas do multiplexador em pinos de saídas de dados.

Nesta construção, não foi explorado o uso de sinais de saídas, algo que pode ser implementado, graças à estrutura criada para os circuitos.

4.1 — Resultado final dos Circuitos e Macros

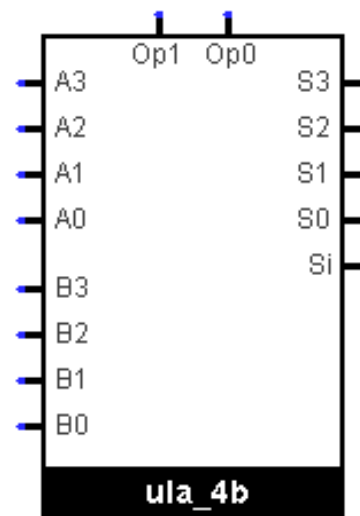
4.1.1 — Circuito Lógico Padrão

Figura 60 — Circuito Lógico Padrão: ULA 4 bits



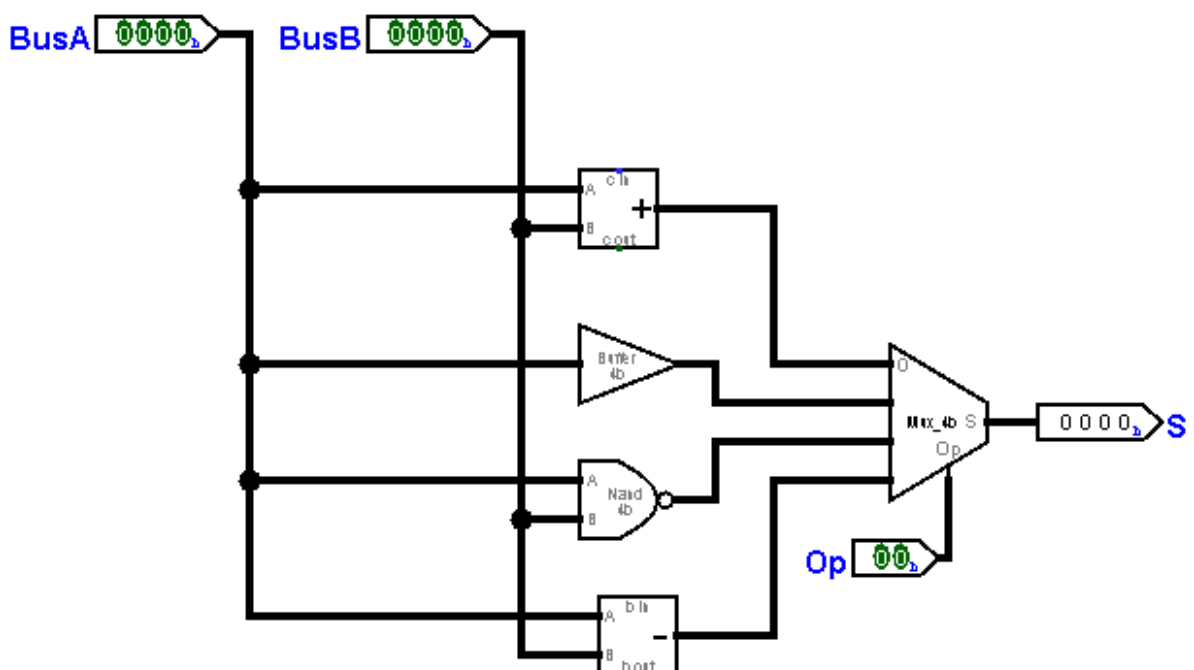
4.1.2 — Bloco Macro Padrão

Figura 61 — Bloco Macro Padrão: ULA 4 bits



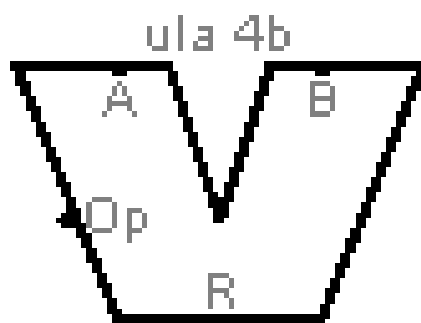
4.1.3 — Circuito Lógico Customizado

Figura 62 — Circuito Lógico Customizado: ULA 4 bits



4.1.4 — Bloco Macro Customizado

Figura 63 — Bloco Macro Customizado: ULA 4 bits



4.2 — Demonstração da ULA

Em seguida, o circuito de demonstração da ULA desenvolvida, permitindo o uso de visores hexadecimais para melhor leitura dos números e um breve sumário das instruções de uso da ULA.

Figura 64 — Macro da ULA Padrão em uso

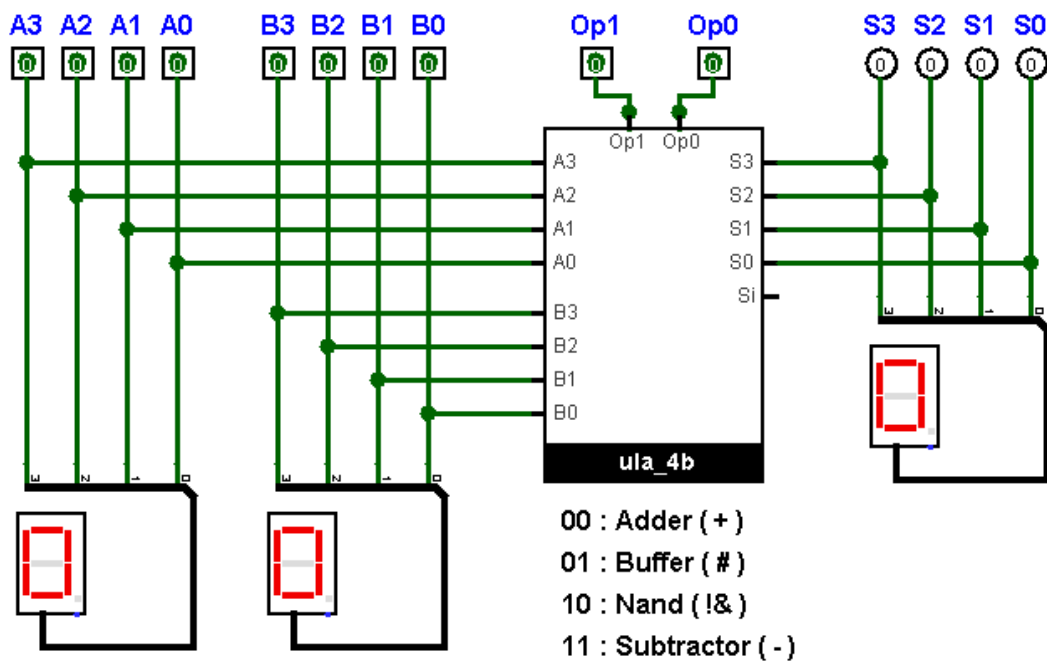
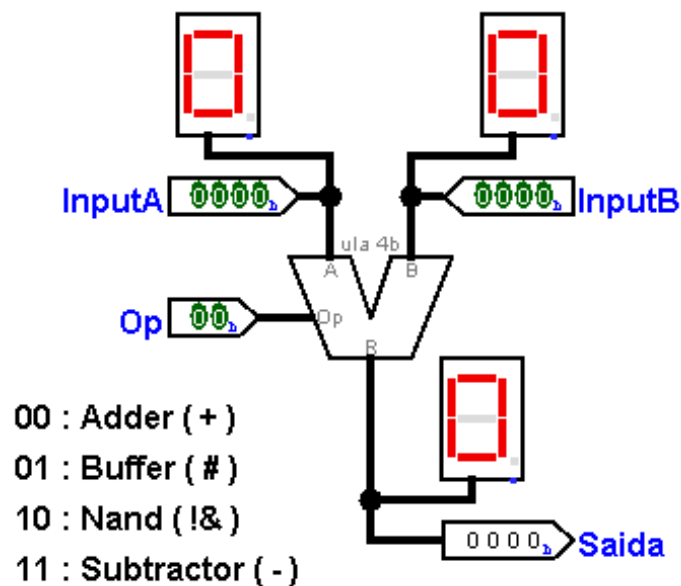


Figura 65 — Macro da ULA Customizado em uso



CONCLUSÃO

A construção da ULA de 4 bits permitiu aplicar, de maneira prática e progressiva, conceitos essenciais de arquitetura de computadores, partindo de blocos lógicos simples até a montagem de uma unidade funcional completa. O uso do Logisim foi fundamental para estruturar, simular e validar cada módulo, possibilitando testes visuais imediatos e facilitando a identificação de eventuais ajustes necessários ao longo do desenvolvimento.

A metodologia adotada priorizou a clareza, a organização e a modularidade. Cada componente — como somadores, subtratores, operadores lógicos e multiplexadores — foi projetado, testado e documentado separadamente antes de ser integrado à ULA. Esse processo sistemático garantiu que o funcionamento geral fosse coerente e que todas as operações respondessem corretamente aos sinais de controle e aos dados de entrada.

Além disso, o modo como a ULA foi construída favorece sua expansão futura. A estrutura modular permite aumentar tanto a largura dos dados (por exemplo, passando de 4 para 6, 8 ou 16 bits) quanto as operações suportadas, que serão exploradas futuramente, bastando replicar ou ajustar blocos já existentes. Essa flexibilidade também facilita sua incorporação em projetos maiores, como uma CPU. Como o circuito foi organizado com cuidado e possui uma arquitetura clara, é provável que seja importado como uma biblioteca no Logisim, servindo como um bloco funcional reutilizável e esteticamente organizado dentro do projeto maior.

Referências Bibliográficas

O entendimento sobre conversões digitais, expressões lógicas, tabela verdade, circuito e portas lógicas, lista de execução, sinais de saída, mintermos/maxtermos, portas básicas e avançadas, unidades lógicas aritméticas, decodificadores, multiplexadores, fundamentos da construção de circuitos digitais, somadores e subtratores e outros pontos vitais, foi possível a partir dos ensinamentos do professor José A. A. Viana (informação verbal, disciplina de Arquitetura de Organização de Computadores, FATEC Rio Preto, 2025).

O entendimento sobre o uso da ferramenta Logisim-Evolution foi feita de forma independente, explorando e testando a ferramenta para praticar, e realizando pesquisas de como utilizar alguns componentes nativos do software.

VOO, Vanessa; DELGADO, Reniê; ALEXANDRO, Daniel. *Sistemas Digitais – Circuitos Aritméticos*. Monitoria SD 2011.2, Centro de Informática – UFPE. Disponível em: <https://www.cin.ufpe.br/~voo/sd/Aula5.pdf>. Acesso em: 11 de novembro de 2025. *Material utilizado para compreensão dos princípios de somadores, subtratores e fundamentos da aritmética digital, servindo como base para estudos posteriores sobre ULAs e operações combinacionais.*

ALLDATASHEET. *Biblioteca online de datasheets de componentes eletrônicos*. Disponível em: <https://www.alldatasheet.com>. Acesso em: 15 de novembro de 2025. *Consultado para obtenção de informações técnicas sobre circuitos integrados utilizados nos exemplos ou estudos relacionados.*

EMBARCADOS. *Portas Lógicas — fundamentos, exemplos e aplicações*. Disponível em: <https://embarcados.com.br/portas-logicas>. Acesso em: 15 de novembro de 2025. *Referência utilizada para visualizar diagramas de portas lógicas em melhor qualidade e complementar a parte conceitual sobre elementos básicos de circuitos digitais.*